

哈 尔 滨 理 工 大 学

毕 业 设 计

题 目： 基于 Web 与 Android 双端的智能
笔记管理系统

院、 系： 计算机科学与技术学院 网络工程系

姓 名： 涂京涛

指导教师： 王莉莉

系 主 任： 杨海陆

2026 年 5 月 27 日

哈尔滨理工大学毕业设计（论文）评语

学生姓名：涂京涛		学号：2204020121	
学 院：计算机科学与技术学院		专业：网络工程专业	
任务起止时间：2025 年 10 月 9 日至 2026 年 5 月 27 日			
毕业设计（论文）题目： 基于 Web 与 Android 双端的智能笔记管理系统			
指导教师对毕业设计（论文）的评语： 该生通过阅读文献对跨平台智能笔记领域进行需求分析，采用 C++、Vue3、Kotlin 等前后端核心技术实现了跨平台智能笔记系统，设计合理，功能完整，具有一定的理论意义和实际应用价值，具备将所学知识用于解决问题的能力。论文逻辑结构合理，图表规范，文字通顺流畅，符合毕业论文撰写规范要求。 指导教师签名：_____ 指导教师职称：_____			
评阅教师对毕业设计（论文）的评语： 论文题目“基于 Web 与 Android 双端的智能笔记管理系统”基于 Android 和软件工程开发技术实现了智能笔记管理系统，选题有一定的应用价值，符合专业培养目标。所开发的智能笔记管理系统功能基本完善，具备将所学知识应用于实践的能力。毕业论文撰写较规范，论文结构层次较分明，逻辑较清晰，语句较流畅，达到本科毕业要求，可以提交答辩。 评阅教师签名：_____ 评阅教师职称：_____			
答辩委员会对毕业设计（论文）的评语： 答辩委员会评定，该生毕业设计（论文）成绩为：_____ 答辩委员会主席签名：_____ 职称：_____ 年 月 日			

教务处制表

哈尔滨理工大学毕业设计（论文）任务书

学生姓名：涂京涛		学号：2204020121	
学 院：计算机科学与技术学院		专业：网络工程	
任务起止时间：2025 年 10 月 9 日至 2026 年 5 月 27 日			
毕业设计（论文）题目： 基于 Web 与 Android 双端的智能笔记管理系统			
毕业设计工作内容： 该系统主要实现一个基于 Web 与 Android 双端的智能笔记管理平台。系统支持用户在 Web 和 Android 设备上进行笔记的基本操作与分类管理，并支持多模态笔记形式。通过内置的 OCR 识别模块，用户上传的图片可自动转换为文本内容；系统还提供笔记自动摘要、关键词提取、全文检索和相似笔记推荐等智能功能，以提升笔记整理效率。平台采用云端数据同步机制，确保用户在不同终端上的笔记内容保持一致。Android 端支持离线编辑，在网络恢复后自动合并数据并解决冲突。 1. 查找相关资料完成系统方案以及框架的设计；（25.10.9——26.1.9） 2. 项目各个模块功能进行详细设计并编写代码；（26.1.10——26.3.20） 3. 测试各个基本模块，根据测试结果完善功能；（26.3.20——26.4.17） 4. 完成毕业论文编写，准备答辩。（26.4.17——26.5.27）			
资料： [1] 夏天生,苏俏媚,郭嘉跃. 基于国内网页端云笔记产品的可用性研究[J]. 包装工程,2025,46(20):445-455. [2] 刘春爽.Android 平台手写笔记系统的设计与实现[D].北京邮电大学,2022. [3] Noor S,Aya M. An AI-Based Interactive Digital Content Application for Document Summarization, Translation, and Productivity Enhancement[J]. Procedia Computer Science, 2025, 262: 123-130.			
指导教师意见： 签名： 年 月 日		系主任意见： 签名： 年 月 日	

基于 Web 与 Android 双端的智能笔记管理系统

摘 要

随着移动互联网的快速迭代，用户的信息记录和知识管理需求不再满足于基本的创建编辑等功能，而是向智能化处理及高效检索方向转变，在智能终端技术的普及下这一需求更是与日俱增。但目前市面上主流笔记软件普遍存在局限性问题。一方面，跨平台同步不稳定的问题影响用户体验；另一方面，非结构化信息的解析处理能力偏弱的问题导致系统响应迟钝；面对海量笔记数据时，检索速度与查询效率上难以满足用户预期。本文针对上述问题，设计并实现双端智能笔记管理系统，以弥补现有应用的不足。

系统采用前后端分离架构，前端包括 Web 和 Android 端。在功能方面，双端均支持笔记的 CRUD 操作以及目录分类等基础功能。在智能处理方面，引入 Elasticsearch 构建全文索引，提高查找效率；通过 MinIO 实现图片和文件等对象资源的统一管理，增强多模态数据管理能力。集成 OCR 识别技术，将图片和文档高效地转化成笔记。融合 NLP 技术，智能生成笔记文本的标签，提升信息的语义结构化程度和可检索性。根据用户的历史行为和检索记录，采用混合推荐算法，实现个性化的笔记推荐。

技术方面，后端基于 C++ 语言的 Drogon 框架构建 RESTful API。选用 MariaDB 作为数据库，基于 Elasticsearch 实现全文检索，通过 MinIO 存储对象，并集成 PaddleOCR 接口实现图片转笔记功能。Web 前端基于 Vue3 开发，Android 端基于 Kotlin 语言进行原生开发。经单元测试与集成测试，系统实现了预定功能。

关键词 笔记系统；Web 应用；Android 应用；全文检索；混合推荐

Intelligent Note-taking System Based on Web and Android Platforms

Abstract

With the rapid iteration of the mobile Internet, the user 's information recording and knowledge management needs are no longer satisfied with the basic functions such as creation and editing, but to the direction of intelligent processing and efficient retrieval. This demand is increasing with the popularity of intelligent terminal technology. However, the current mainstream note-taking software on the market generally has limitations. On the one hand, the problem of unstable cross-platform synchronization affects the user experience ; on the other hand, the problem of weak analytical processing ability of unstructured information leads to slow system response. In the face of massive note data, it is difficult to meet user expectations in terms of retrieval speed and query efficiency. In view of the above problems, this paper designs and implements a two-terminal intelligent note management system to make up for the shortcomings of existing applications.

The system adopts the front-end and back-end separation architecture, and the front-end includes Web and Android. In terms of function, both sides support basic functions such as CRUD operation of notes and directory classification. In terms of intelligent processing, Elasticsearch is introduced to build full-text index to improve search efficiency. Through MinIO, the unified management of object resources such as pictures and files is realized, and the multi-modal data management ability is enhanced. Integrate OCR recognition technology to efficiently convert pictures and documents into notes. Integrating NLP technology, the label of note text is generated intelligently, and the semantic structure and retrieval of information are improved. According to the user 's

historical behavior and retrieval records, a hybrid recommendation algorithm is used to realize personalized note recommendation.

In terms of technology, the back-end builds a RESTful API based on the Drogon framework of the C++ language. MariaDB is selected as the database, full-text retrieval is realized based on Elasticsearch, MinIO is used to store objects, and PaddleOCR interface is integrated to realize picture transfer note function. The Web front-end is developed based on Vue3, and the Android side is natively developed based on the Kotlin language. Through unit test and integration test, the system has realized the predetermined function.

Keywords Note-taking system, Web application, Android application, full-text search, hybrid recommendation

目 录

摘要.....	I
Abstract.....	II
第 1 章 绪论	1
1.1 课题目的和意义.....	1
1.1.1 课题目的.....	1
1.1.2 课题研究意义.....	2
1.2 国内外研究现状分析.....	2
1.2.1 国内研究现状分析.....	2
1.2.2 国外研究现状分析.....	3
1.3 论文的主要内容.....	5
第 2 章 系统需求分析	7
2.1 可行性论证.....	7
2.1.1 系统目标.....	7
2.1.2 可行性分析.....	7
2.2 系统需求分析.....	8
2.2.1 系统功能需求概述.....	8
2.2.2 系统功能需求分析.....	9
2.2.3 其它非功能需求分析.....	11
2.3 本章小结.....	11
第 3 章 系统设计	12
3.1 系统概要设计.....	12
3.1.1 系统体系结构设计.....	12
3.1.2 系统功能结构设计.....	13
3.2 系统功能模块设计.....	14
3.2.1 用户认证模块.....	14
3.2.2 笔记管理模块.....	15
3.2.3 文件夹管理模块.....	18
3.2.4 文件与 OCR 模块.....	19
3.2.5 标签管理模块.....	20
3.2.6 搜索模块.....	21
3.2.7 推荐模块.....	22
3.3 数据库设计.....	23
3.3.1 数据库概念模型设计.....	23

3.3.2 数据库物理模型设计	26
3.4 本章小结	29
第 4 章 系统实现与测试	30
4.1 核心算法与机制实现	30
4.1.1 基于用户行为与标签的推荐算法	30
4.1.2 ES 全文检索实现	32
4.1.3 系统其他机制	33
4.2 系统实现效果	34
4.2.1 Web 端实现效果	34
4.2.2 Android 端实现效果	43
4.3 系统测试	49
4.3.1 单元测试	49
4.3.2 集成测试	51
4.4 本章小结	52
结论	53
致谢	54
参考文献	55
附录	57

第 1 章 绪论

1.1 课题目的和意义

1.1.1 课题目的

移动互联网如今无处不在，手机、平板等各类智能终端也越来越普及。根据 Gartner 2025 年调研数据，85%的企业知识资产处于未有效管理状态，员工平均每周浪费 7.3 小时在信息检索和验证上。全球笔记应用市场规模从 2024 年的 95.4 亿美元增至 2025 年的 110.2 亿美元，年增长 15.6%；百度文库 AI 月活用户近 9700 万，其多模态 AI 笔记支持视频内容一键生成结构化笔记和思维导图。据“知识结构化”产业白皮书数据，抖音、B 站等平台知识短视频日均播放量达 280 亿次，但完播率不足 35%，62%的用户自评“学完不会用”，碎片化内容缺乏体系化串联的矛盾日益突出。可见，用户对于碎片化信息记录和知识管理方面的需求明显增长了，呈现出了多设备、多场景、多模态化等形式。然而，目前主流笔记工具或应用大多是以某一终端为基础来构建开发的。功能模块根据终端来设计，彼此之间相互隔离，难以支撑跨终端的无缝衔接与高效管理，缺乏终端之间交互共享的有效支持。国内学者提出，现有云笔记产品在可用性上仍然存在跨平台同步不稳定或有延迟，以及结构化信息解析能力不足和格式不兼容等问题^[1]。同时，大部分系统在手机等移动端只提供了简单的浏览功能，缺少和桌面端对等的智能化处理能力^[2]。个人笔记数据不断增长，私人备忘录内容也相应增多，仅使用传统人工目录分类和基础关键词搜索的管理方法，很难以满足用户即时语义检索和知识复用的需求^[3]。

本文主要目标是设计并实现一套基于 Web 与 Android 双端的智能笔记管理系统，以解决上述问题。系统运用前后端分离模式，依靠统一后端服务实现多终端数据集中存储管理与数据在双端的状态同步一致性。系统在功能上除了基础的文本笔记创建、编辑和多级分类外，还集成了 OCR 技术。通过运用基于深度学习的版面分析和文本识别方法^[4]，将图片及 PDF 扫描文档自动转换成可以编辑的结构化笔记。同时引入了预训练语言模型和重要短语预测技术^[5]，对笔记文本进行语义标签生成，以此增强信息的结构化程度和可检索能力。系统也部署了分布式全文检索引擎，对检索速度和准确度进行优化，提高海量笔记场景下的查询效率^[6]。结合了用户的使用情况，即通过用户行为上下文来实现混合推荐，提高了个性化定制化服务能力^[7]。本文旨在构建融合多模态感知、跨平台协作与智能分析的一体化笔记系统，为个人知识管理提供高效可靠的解决方案及相关的技术支持。

1.1.2 课题研究意义

本课题的研究意义主要体现在三个方面，分别是用户体验优化、工程实践价值和技术方法推广。

就用户体验而言，各类丰富的信息使用户对笔记系统的要求已经由基础记录转变为更高效的智能整理。现有工具在弱网环境或离线状态下，存在数据统一同步一致性不足、非文本数据结构化可读化能力欠缺等问题^[8]。本文提出了若干策略以实现不同设备之间的实时性管理、离线存储功能。在原有基础上设计了多端协同和异步处理方式。同时把 OCR 与 NLP 技术结合，实现对文本、图像等不同模态数据的一体化管理及语义标注。这一设计可提高用户整理资料、回溯信息以及获取知识的效率。

在工程实践价值中，本文从实际应用角度出发，涉及横跨 Web 前端构建、Android 客户端原生开发和高并发后端服务建立等多个技术领域。系统的技术集成度较高，涉及内容也较丰富。系统的后端使用 C++ 语言的 Drogon 框架来建立 RESTful API 服务，并且结合 MariaDB、Elasticsearch、MinIO 这些组件共同构成分布式存储和计算处理平台。该过程也说明了多种技术栈结合是可以实现的。此系统通过运用消息队列进行非实时异步协调调度、对增量式数据进行同步，并且还利用了 Docker 容器化部署等工程技术，能够为同类复杂系统的设计和 implement 提供拓展的参考经验^[9]。

技术方法推广这一方向上，系统不仅运用第三方 OCR 接口生成笔记，而且通过 NLP 技术为笔记生成标签，还根据用户行为生成个性化笔记推荐。这一系列智能技术扎实了笔记管理流程，形成了从笔记的信息采集到语义结构化再到主动推送的闭环。这一方案为相关的技术路线提供了完全可复用的技术参考，具有直接的可移植价值。此外，系统中采用的模块化设计的核心模块大部分基于开源生态实现，具备可扩展性与可维护性的优势，未来新增特性与优化性能具有了扎实的基础。

1.2 国内外研究现状分析

1.2.1 国内研究现状分析

在互联网时代下，随着信息技术不断更新，数字化电子笔记工具已经成为个人知识管理和协同办公的主要工具。日常生活中，人们记录、保存和分享各类信息的主要工具已离不开笔记软件。在国内主流平台，笔记软件已形成较完整的产品模式。以有道云笔记、语雀和石墨文档为代表的各类产品，形成了比较完整的生态。有道云笔记和印象笔记主要重视个人知识管理及多端同步办公，允许用户利用 Web 网页端完成截图、OCR 文字识别以及 PDF 文档扫描等操作。已有文献对有道云笔记和印象笔记可用性进行了整体评估。语雀自定位为建立结构化知识库的平台，重视对文档进行

组织化分类管理和团队知识沉淀。飞书文档和石墨文档则主要关注在线文档协同编辑场景，支持多人进行实时协作。上述产品在功能丰富度上已经取得明显进展并且日趋完善，但存在非结构化信息的深度解析和跨端数据一致性保障等许多有待解决的问题，工程界和学术界仍然在进行探索^[10]。

可见国内在智能笔记系统领域的研究，主要集中在三个方向：多模态信息处理、智能文本分析以及系统架构优化。

相关研究分析多模态异构信息的输入处理，提高了信息采集录入效率和识别准确度。移动设备的摄像功能不断更新，依靠 OCR 技术进行图片文本识别逐渐成为笔记系统的主要功能部分并得到广泛运用。有关研究通过运用图像预处理算法提出改进方案，一定程度上提高了复杂场景中中文文本识别的准确度，使图像内容转化为可编辑文本的可用性得到提升。当用户因各种各样情况，如手机厂商迭代导致硬件落后、平板不考虑新版，而导致移动端的资源受到限制时，如何获取较高识别精度并降低响应延迟，仍然是需要解决的主要问题。

在国内 NLP 的应用中，研究主要集中于关键词提取、自动摘要和文本分类等功能上。由于 TF-IDF、TextRank 以及融合模型作为底层的轻量级算法具有实现复杂度低、计算开销低、成本小等优势，在实际系统中得到了广泛应用。这些应用主要用来提高笔记内容的结构化、可加工程度和检索效率。同时，近几年预训练语言模型的研究发展，一些研究工作开始尝试把 BERT 等这类基于深度神经网络的语言模型用于语义表征和内容推荐。有文献综述指出，这些预训练语言模型在主要短语预测，标签生成任务上使用很合适，表现出明显优势。不过在实际工程部署过程中，仍然需要在推理带来的计算资源消耗和接口响应效率之间进行权衡。

在系统架构和数据同步角度，国内笔记系统大多使用云端集中存储和各终端访问的方式来进行操作。大部分应用采用传统请求响应方式进行数据同步。并且通过时间戳覆盖或后写入优先等方式处理冲突问题。此类方法实现较简单，复杂程度较低，但在使用场景复杂化时仍会出现数据不一致的情况。为了对非结构化信息进行有效组织，一些研究从内容节点角度分析在线文档信息组织的灵活性^[11]，提供了笔记系统数据结构设计的理论依据。此外，面向特定场景的笔记系统设计也取得了一定进展：Android 平台的手写笔记应用可采集和回放书写轨迹；使用文本相似度对个人笔记进行管理，给语义检索带来了新的思路。

综上所述，国内在智能笔记系统工程实现上已经具备一定基础并且取得部分成果。在多模态数据深度融合、跨端数据一致性保障和智能化处理能力等方面仍需进一步突破。

1.2.2 国外研究现状分析

和国内研究相比，国外在多模态信息处理和智能笔记系统方面的研究

起步较早。有关文献在理论模式、系统结构和商业化产品上研究均较成熟。在商业化应用产品上, Notion、Evernote、Microsoft OneNote 和 Obsidian 等软件占据了大部分全球笔记软件市场。

Notion 依靠模块化块编辑和高度可定制的工作空间而知名, 支持笔记、数据库、看板和日历等多种视图进行无缝整合。Notion 还被广泛评价为优秀的知识管理和团队协作主要平台。基于 Notion 进行的教学模式研究说明, 模块化笔记平台在结构化课程设计和知识组织上拥有很大潜力^[12]。Evernote 作为早期云笔记产品的主要代表, 依靠强大的网页剪藏功能、跨平台数据同步共享和标签分类管理获得了大量用户的认可。虽然 Evernote 长期维持着市场份额, 但用户体验和功能设计也引起了专门的学术研究^[13]。微软旗下的笔记软件 OneNote 和自家的 Office 生态进行深度集成。OneNote 拥有手写笔记、音频录制、多媒体包括图片等多种形式的嵌入功能, 因此在教育和办公场景中应用广泛。Obsidian 则代表着本地优先笔记软件的新一代发展趋向。该软件利用 Markdown 纯文本格式和双向链接方式, 注重用户对个人私有数据的完全控制权。该软件在开发者、学术用户群体里获得广泛接受, 得到了较高的认可度。已有研究运用 DIKW 模型对 Obsidian 在个人数字化存档中的应用进行分析, 指出该工具在学习曲线和信息过载上存在一些不足之处^[14]。

国外学者在学术研究上对多模态数据表示和数据融合进行大量探索, 提出了把文本、图像和语音等数据映射到统一特征空间的方法, 给跨模态检索和统一信息管理提供理论基础。这类方法将不同类型的数据在语义上联系起来, 用户信息的检索和组织扩展到多种模式, 让系统达到更高的智能化程度。

在 OCR 和 NLP 技术上, 国外普遍使用依靠深度学习的端到端模式。OCR 识别领域中, 利用卷积神经网络 CNN 和序列建模方法的模型, 在进行复杂版面分析、多语言识别、低质量图像处理时具有较高的准确率。最新研究说明了运用深度学习的印刷文档版面分析和 OCR 系统的技术路线图。这一研究引入了笔记系统中图像转文本功能的新方法, 属于很好的技术支持。NLP 技术在文本分析领域已非常成熟, 包括但不限于情感分析计算技术、主题建模分类技术和文本生成摘要技术等^[15]。该技术给智能笔记系统的语义理解模块提供了扎实的理论支持, 指出明确的智能化研究方向。

国外研究在系统架构设计上主要采用了云端协同的方式, 将计算密集型运算负载部署在云端, 通过标准化的 RESTful 或 GraphQL 接口向客户端提供服务。该架构可以减轻终端设备的计算负荷, 提高系统的可扩展能力, 有利于对系统特性进行扩展。已有文献说明利用人工智能作为驱动力的交互式文档处理应用设计方法、依靠资源导向模式的移动个人知识管理平台集成方案^[16]。上文所述内容为多端智能笔记系统的工程实现提供了直接参考。在数据同步这一方面看, 若干先进的系统运用操作转换或者无冲突复

制数据类型这类分布式一致性算法，以处理不同终端之间进行协同编辑时出现的数据一致性问题^[17]。在移动端协作场景里，有的研究把基于WebSocket的实时同步方式应用到了笔记应用开发上，形成了流畅的在线协作平台。

纵观国外研究，智能笔记系统的算法研究和系统架构设计上相对国内更成熟，已形成较为完整的技术模式。但其复杂的实现过程通常需要较多的计算资源和系统成本。在具体应用上，系统性能、资源使用情况和用户使用感受之间的平衡问题仍有探索必要。

1.3 论文的主要内容

本文以智能笔记管理系统为切入点，主要研究跨终端环境下笔记数据的统一管理模式。以此为基础加上智能化处理模块来提高笔记记录、整理、查询的能力。

从系统功能角度分析，本文改善用户日常笔记操作的全过程体验。相比旧式笔记应用只提供浅层的文本录入和目录归类，本系统可以对笔记进行增、删、改、查以及多级文件夹的组织。使用Elasticsearch全文检索引擎，使用户可以在大量的笔记中快速找到需要的文档。为达到检索效率和精度之间的平衡，在系统设计之初采用相关调优策略。为有效地降低手动输入的成本，内置OCR识别功能，可接受图片或者PDF扫描件作为输入，将其中的文字内容自动提取并转化为结构化笔记。融合NLP技术，对笔记文本进行语义分析并自动产生标签，完成笔记的智能化分类和管理。参考PLM在标签预测任务上取得的研究成果，提升生成的准确性、可读性。

在智能化服务方面，本文进一步研究怎样根据用户的使用习惯来实现个性化的推荐。系统依据用户浏览、点赞、收藏等行为的标记以及笔记标签信息形成用户兴趣画像，进而推送相似的笔记。推荐算法吸收了上下文感知的混合推荐思想，相比较于单纯的按照时间逆序排列，更符合用户的兴趣偏好。考虑到新用户的历史行为较少，系统采用了热门标签加基础标签的方式解决冷启动问题，将推荐功能覆盖到了用户生命周期的所有阶段。

系统的总体架构设计为前后端解耦的多端协作体系。后端利用C++的Drogon框架来实现高并发的RESTful服务，完成业务逻辑和数据控制的业务。Web端用Vue3框架创建单页应用，提供直观且流畅的交互界面。Android端用原生Kotlin语言开发，实现移动场景下轻便性同时保留原生质感。系统依靠标准的RESTful接口达成多端数据的统一，并运用JWT鉴权保障用户的隐私和访问控制。此外采用MariaDB做数据持久化；Elasticsearch创建全文检索；MinIO存储文件对象。从数据持久化来说，满足系统对于结构化、索引和非结构化文件等多种数据类型的需求。

系统运行过程中对异步任务调度、多端数据一致性等值性保证等问题

进行了研究。对于文件上传、OCR识别和索引重建这些I/O密集型或计算密集型业务，采用消息队列驱动的异步调度方式，避免主事务流阻塞，提高了全局响应速度以及吞吐量。设计详细的版本控制和增量同步机制保证用户在不同的终端上进行操作时得到一致结果，提高系统稳定性。设计吸取了实时协同笔记系统中使用WebSocket实现同步的工程经验。

本文接下来的章节会依照以上内容层层展开。第一部分分析系统功能需求；第二部分提出系统总体架构及各功能模块设计；第三部分根据具体的实现方式论述关键技术路线以及系统测试结果。最后是对全文研究工作进行总结，展望未来改进方向。

第 2 章 系统需求分析

2.1 可行性论证

2.1.1 系统目标

本文设计的智能笔记管理系统旨在为用户提供一个可以进行多端协同、智能处理和高效检索的综合信息管理平台。系统在传统笔记系统之上，进一步改善了用户在信息记录、整理和利用的整体体验。

在基础功能上分析，系统应当允许用户进行基本的笔记创建、编辑和删除管理，同时利用多级文件夹结构来实现对笔记内容的层次化组织，使用户可以更清晰地管理个人知识体系。系统还应当具有全文检索功能，用户可以使用关键词快速定位目标笔记，避免在大量数据中逐条查找，从而提高信息检索效率。

功能扩展角度来看，系统需引入 OCR 识别技术，使用户能够通过上传图片或者文档，将其中文本内容自动提取并生成笔记，降低信息录入的成本。结合 NLP 技术，系统能够对笔记内容自动生成标签，实现了非结构化数据的结构化处理，从而让笔记内容更容易被分类和检索。

有关智能服务，系统利用对用户浏览、点赞和收藏等行为数据的分析，形成用户兴趣模型。并且依靠标签信息进行了个性化笔记推荐。该功能依据用户的使用习惯自动提供相关笔记，减少用户获取信息时间成本，并提高系统的实用性和智能化水平。

在系统运行方面，本文还致力于实现多端数据的统一管理与同步。用户可以经 Web 或 Android 端访问系统，所有操作均会通过统一的后端服务进行处理，以此来确保数据的一致性和安全性。利用异步处理方法来完成文件上传、OCR 识别、搜索索引更新等耗时操作，该方式能够防止系统主流程发生阻塞，改善整体的响应性能。

2.1.2 可行性分析

2.1.2.1 经济可行性分析

系统的开发主要依靠开源技术，整体开发成本较低。后端采用基于 C++ 语言的 Drogon 框架。Web 前端采用 Vue3 框架。移动端基于 Android 原生开发。各种开发工具及框架均可以免费获得并进行运用，降低软件开发的成本。在数据存储上使用关系型数据库 MariaDB，引入 Elasticsearch 实现全文检索功能，并使用 MinIO 存储文件对象。上述组件均属于开源软件，部

署成本可以控。

在开发周期方面，系统作为本科毕业设计项目，开发工作主要在数月内完成，开发规模较适中，功能边界明确，不涉及大规模商业系统的复杂业务逻辑，人力成本和时间成本都处在可控范围里。可以依靠本地运行或者云服务器进行部署，硬件资源需求较低，进一步降低系统运行成本。

总体而言，系统在开发成本、部署成本和维护成本上均表现出较好的可控性，从经济角度出发具有较强的可行性。

2.1.2.2 技术可行性分析

该系统在技术选型上对系统性能、开发效率和可扩展性等因素进行了充分考量，整体技术方案成熟可靠，并且拥有较好的实现基础。

在后端开发上，系统运用 Drogon 框架构建 RESTful 服务。Drogon 框架基于 C++，Linux 平台在 epoll 加持下高性能异步处理能力优秀，可有效应对高并发请求环境。并且框架提供对象关系映射 ORM、路由管理、中间件等功能，能够满足系统在数据处理和接口服务上的要求。数据存储方面，MariaDB 作为成熟的关系型数据库管理系统，它可稳定对用户数据和业务数据进行持久化存储。Elasticsearch 用于实现全文检索功能，倒排索引结构明显提高了查询效率，可满足大规模笔记数据的快速检索要求。

前端实现过程中，Web 端利用 Vue3 框架来建立单页应用。系统结合了组件化开发和 Pinia 状态管理，可以对复杂界面进行高效的开发和维护。Android 端使用 Kotlin 进行原生开发，并且结合 MVVM 架构模式，实现界面和数据逻辑的解耦，使系统具有更好的可维护性和扩展能力。

在扩展技术上，系统把 OCR 识别和 NLP 进行集成。OCR 识别利用成熟的第三方 PaddleOCR API 来完成，避免了自行实现复杂算法，提高开发效率并保证识别准确率。AI 标签生成利用大模型接口进行语义分析，提高智能化水平。系统使用 MinIO 存储文件，利用 S3 兼容接口完成文件的上传和访问，技术实现成熟稳定。

系统运行机制上，针对文件上传、OCR 处理及搜索索引更新等耗时的操作，采用异步处理方式。该方式有效避免阻塞主线程，提高系统的响应性能。利用 JWT 认证来验证用户身份，从而保证访问安全。

总体来看，系统各类技术均属于当前较成熟的处理方式。在实际开发过程中已得到普遍使用，可达到系统功能实现和性能要求，技术可行。

2.2 系统需求分析

2.2.1 系统功能需求概述

根据系统设计目标，本文设计的智能笔记管理系统主要有 8 个模块。围绕用户信息管理、笔记管理、文件夹管理、全文搜索、文件管理、OCR

识别、标签管理和推荐服务等进行。各功能需求简要介绍如下。

用户管理功能主要用于实现用户的注册、登录和身份认证。系统利用账号密码进行用户身份校验，依靠 JWT 机制维持用户登录状态。用户登录系统后，可获取个人信息并进行基本信息管理，从而保障系统访问的安全性和个性化定制服务的实现。

笔记管理功能主要负责进行笔记的创建、编辑、删除和查询操作。用户可实时编辑笔记内容，同时系统通过自动保存等机制避免数据丢失。也支持笔记进行公开或私有设置。系统应允许用户对公开笔记进行浏览、点赞、收藏等交互操作，增强用户之间信息共享和交流。

文件夹管理功能用于笔记进行层次化组织。系统支持多级文件夹结构，用户可进行创建、修改和删除文件夹操作。并将笔记归类至不同目录中，从而对大量笔记内容进行有效管理，提高信息整理的效率。

全文搜索功能基于 Elasticsearch 实现。用户可以依靠关键词对笔记标题、内容、摘要和标签等字段进行检索。系统会返回相关度较高的搜索结果，并进行高亮显示，从而帮助用户快速定位目标信息。

文件管理功能主要用于图片和附件的上传与管理。系统通过 MinIO 存储文件对象，用户可上传文件到服务器。并在笔记中以 Markdown 语法引用相关资源。系统提供文件状态查询功能，用户可以借此了解上传进度和处理结果。

OCR 识别功能主要用来处理图片或者文档中的文本信息。用户可上传图片或 PDF 文件，系统通过调用 OCR 服务对内容进行识别。并将识别结果转换为文本形式笔记，减少人工输入，提高信息录入效率。

标签管理功能实现笔记内容的语义分类。系统支持用户手动生成标签，基于 NLP 技术自动生成，实现笔记内容的结构化组织，进而提高后续的检索和推荐效果。

推荐功能会分析用户的行为数据和标签信息，形成用户兴趣模型。继而向用户推荐有关的笔记内容。为新用户和老用户制定不同策略，保证推荐效果同时，也提高系统的智能化程度。

2.2.2 系统功能需求分析

该系统的主要参与者是用户，图 2-1 为系统用户的用例图。系统主要由用户认证模块、笔记管理模块、文件和 OCR 模块等构成。后文详细论述模块具体功能。

用户认证模块功能包括用户注册登录、退出登录和查询/更新个人信息等操作。用户在注册时需提供用户名、密码以及邮箱等基本信息，系统前端应当对这些数据提供基本的数据安全性合法性建议，如用户名长度检验、密码强度检验、邮箱格式合法性检验。系统后端会对这些信息进行合法性校验，并且完成数据存储。系统在登录时会对用户身份进行验证，并且生

成相应的认证令牌。用户在登录之后，能够查询个人的相关信息并且进行必要的更新操作，如更换头像，修改用户名与密码等。该功能主要满足用户个性化需求，给系统其他模块提供基础的身份认证支持。

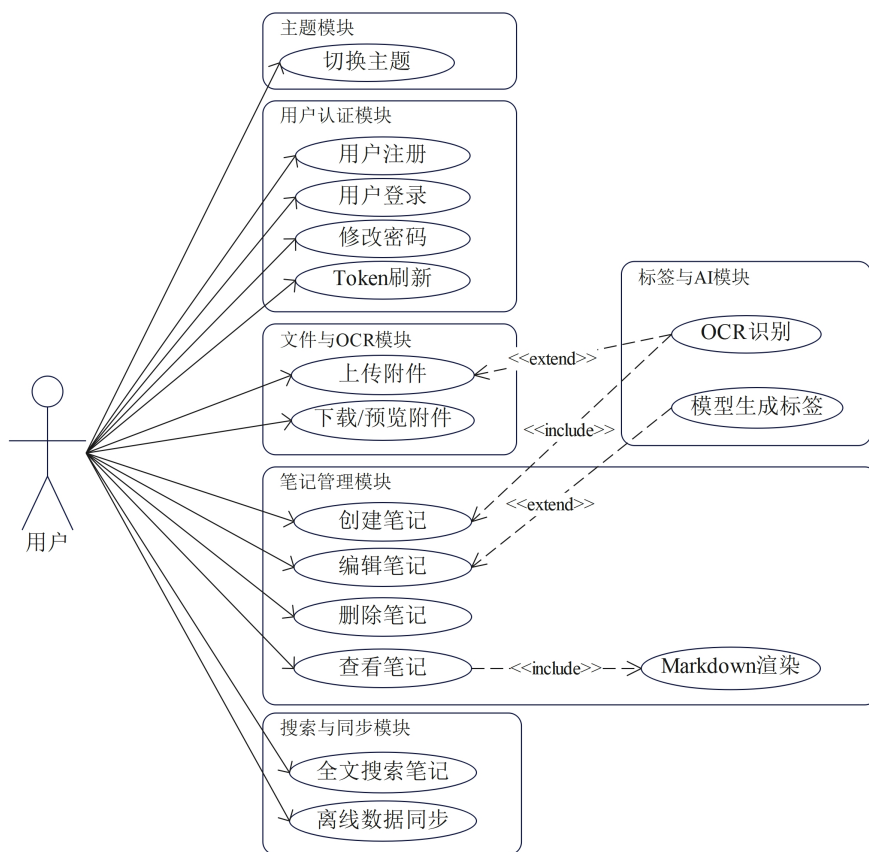


图 2-1 用户用例图

笔记管理模块功能主要包括对笔记进行创建、编辑、删除和查询等操作。用户可新建笔记并附带标题、内容等笔记信息。为提升用户体验，系统前端应当具有 Markdown 渲染的功能。并支持图片等非结构化数据的展示。用户在编辑笔记时需存在自动保存以及手动保存机制来提高使用的安全性。考虑到用户的使用习惯，也应当支持快捷键保存。在修改笔记时，系统会实时地更新内容，并同步到后端持久化存储。用户也可对笔记进行删除或恢复操作，以实现数据的灵活管理。系统应提供用户公开笔记的功能，对于公开笔记，系统应当具备基础的查看、点赞和收藏功能。文件夹管理模块功能主要用于笔记分类存储。用户可创建多级文件夹，对文件夹进行重命名或删除操作。删除文件夹时会删除文件夹中笔记。用户可移动笔记到指定的文件夹中，并利用树形结构展示目录关系，直观地管理笔记的层级结构。

全文搜索模块使用户可通过关键词进行搜索，以此快速定位笔记内容。

系统使用全文检索引擎对笔记数据建立倒排索引，用户发起搜索请求后系统返回相关结果。考虑到用户使用体验，搜索结果中对关键词进行高亮显示。系统应开放公开笔记检索的功能，以满足用户仅浏览的需求。

文件管理模块功能主要体现在用户图片和附件的上传与统一管理。系统在进行文件上传时应使用异步处理方式，提供实时查看文件的上传状态的功能。用户在文件上传完成后，可将文件链接以 Markdown 语法插入到笔记中，从而达到多媒体内容整合目的。

OCR 识别模块主要将图片或文档中文字信息转化为笔记内容。用户提交识别请求后，系统会调用外部 OCR 服务进行处理，识别完成后会生成对应的笔记。该功能降低了人工操作要求，同时提高了系统对非结构化数据的处理能力。标签模块主要功能是为笔记智能生成标签和获取火热标签。系统利用 LLM 为用户的笔记内容生成预定的标签，从而提高了检索与整理效率。热门标签获取利用 ES 聚合查询实现，是本模块辅助功能。标签模块作为推荐算法的基础，合理设计能有效提高推荐精度。

推荐模块利用用户历史行为数据和笔记的标签信息，给用户进行个性化内容推荐。系统按照用户的兴趣来动态调整推荐结果，使用户可更高效地获取相关信息，进而提高整体的使用体验。此外，新用户的行为数据稀少，在新用户推荐时采用了不同算法以及兜底策略保证推荐功能稳定。

2.2.3 其它非功能需求分析

系统在正常运行情况下，应拥有良好的响应性能。正常请求响应时间需控制在 2 秒以内。在并发访问场景中，应支持一定规模的用户同时访问而不出现明显性能下降。

系统安全性涉及到用户个人隐私。系统需对用户账户信息进行严格保护，用户密码通过加密方式进行存储，防止敏感信息泄露。用户在访问系统时，利用 JWT 机制来完成身份认证，防止出现未授权访问的情况。主要操作需进行权限控制，来保证数据安全，如用户不可修改公开笔记，不可为公开笔记重新生成标签等。

系统需具备良好的用户交互体验，界面设计应当简洁直观，操作流程清晰。Web 端可运用响应式布局来适配不同设备，Android 端应具备便捷的操作方式，从而降低用户学习成本，提高系统的使用效率。

2.3 本章小结

本章进行了系统整体需求分析，论证了系统经济和技术两维度的可行性，明确了开发成本可控、技术方案较成熟。通过用例分析和模块细化，梳理了主要功能需求，定义了性能、安全、易用性等非功能需求，为后续系统设计提供依据。

第 3 章 系统设计

3.1 系统概要设计

3.1.1 系统体系结构设计

本系统采用前后端解耦的多终端协同架构。整体由 Web 客户端、Android 客户端及后端服务三部构成，并辅以多种基础设施组件。架构遵循“表现层-服务层-数据层”的垂直分层规约，系统架构如图 3-1 所示。

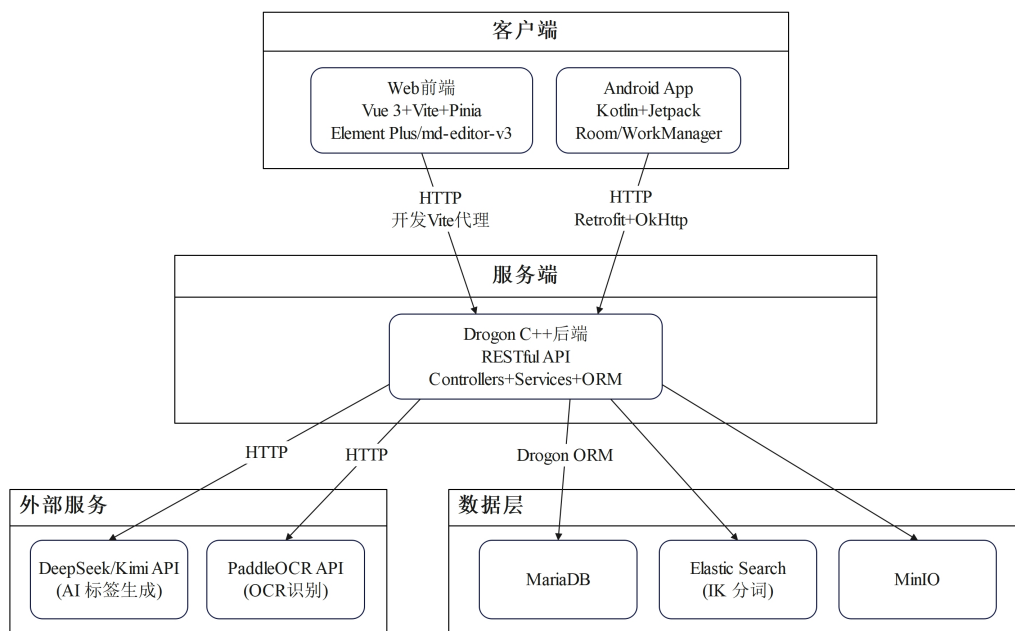


图 3-1 系统架构图

架构图中系统前后端分离清晰可见：客户端层、服务端层、数据层和外部服务四大板块。后文将详细论述系统核心技术路线与机制细节。

客户端层主要分为两种交互入口。Web 端基于 Vue3 框架构建单页应用，主要负责页面渲染和用户交互；Android 端运用 Kotlin 进行原生开发，并结合 MVVM 结构分离了视图和逻辑，增强了可维护性。两端均通过 HTTP 协议调用后端 RESTful API，统一数据访问与业务处理由此实现。

服务端层基于 Drogon 构建形成高性能框架，该层主要负责的是客户端请求的处理工作和业务逻辑的执行，并且协调各类外部服务。内部依控制层、业务逻辑层及数据访问层的结构划分，达到清晰的职责分离。控制层负责进行路由分发和参数校验，业务层承载核心逻辑的处理，数据层则由

ORM 和数据库交互。

数据层采用了异构存储矩阵。MariaDB 作为关系型数据库负责结构化数据的持久化；部署 Elasticsearch 构建倒排索引加快了全文检索；MinIO 则遵循 S3 协议用来存储图片等非结构化对象。此外，系统通过异步调用外部 PaddleOCR 服务和 Deepseek LLM 接口，完成了文本识别与标签选取。未来避免主流程被阻塞，比较耗时操作如文件上传与索引重建都采用异步机制，进而优化系统的整体吞吐。

3.1.2 系统功能结构设计

依据千问的功能需求分析，系统可划分为若干职责明确的模块。具体而言，涵盖用户管理模块、笔记管理模块、文件管理模块、文本识别模块、智能推荐模块、系统管理模块、文件夹管理模块、标签管理模块、数据统计分析模块。各个模块之间保持了彼此解耦的状态并协同运作，共同支撑起了笔记管理的完整业务闭环。系统的功能结构如图 3-2 所示。

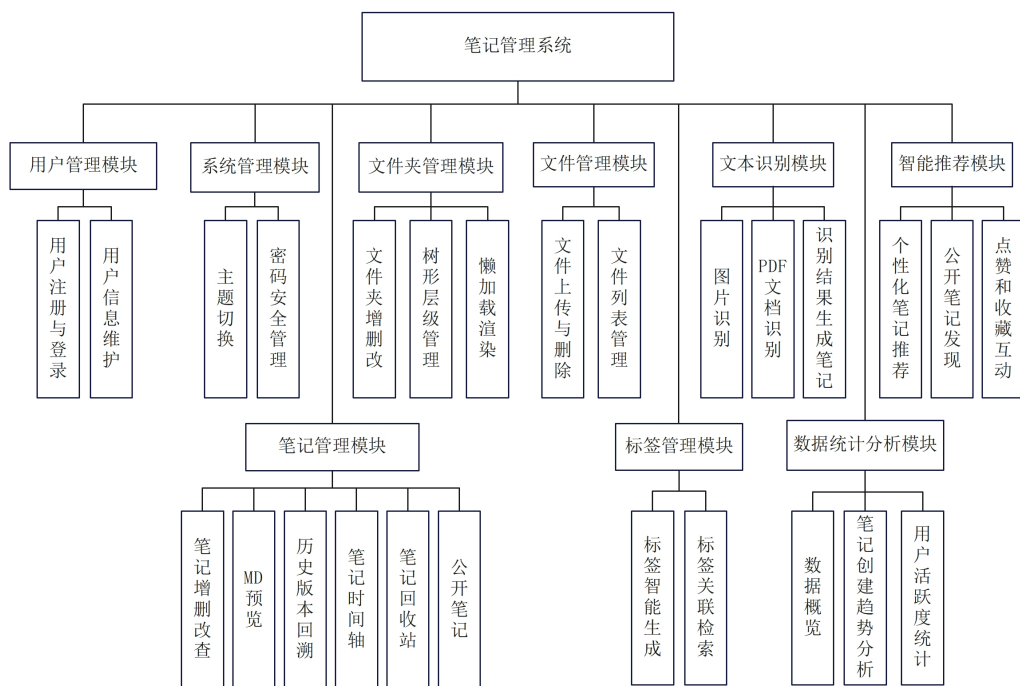


图 3-2 系统功能结构图

用户管理模块拥有用户注册、登录认证和个人信息维护功能，主要用于系统认证用户的身份。笔记管理模块承接核心业务逻辑，主要统筹处理笔记的新建、删除、更新、查询、Markdown 实时预览、历史版本回溯及公开笔记等全生命周期操作。围绕笔记这一实体，文件可在笔记中嵌入，因此文件管理模块负责进行附属资源的上传、删除和列表化管理。文件夹管

理模块实现了文件夹的增删改、树形层级和懒加载渲染，属于空间层级。标签管理模块则完成了标签的智能生成、关联检索，属于是语义归类。两个模块分别从不同的维度提高了笔记的组织效率。文本识别模块提供图像及 PDF 文档转化为笔记的功能，使得笔记的多模态采集入口拓展了。智能推荐模块按照用户行为偏好进行提供了个性化笔记推荐、公开笔记发现及处理点赞收藏互动都功能。此外，系统在管理模块也兼顾主题风格切换与密码安全策略配置，保障了可定制性与安全性；数据统计分析模块展示数据概览、笔记创建趋势和用户活跃度统计，这些内容为迭代优化提供量化依据。各模块由统一接口进行数据交互，共同构成完整的系统功能体系。

3.2 系统功能模块设计

本节以概要设计为基础，逐次详述各功能模块的具体实现。依据模块化设计原则，将业务逻辑解耦为多个相对独立的功能模块，彼此间由统一的 RESTful 接口进行通信，以此达成高内聚与低耦合的系统结构。

3.2.1 用户认证模块

用户认证模块注重注册、登录及身份核验诸项功能，构成系统安全机制的根基。基于 JWT（JSON Web Token）实现无状态登录，用户成功登录后，系统签发访问令牌，后续请求须于请求头部携带该令牌以供鉴权。该模块提供的典型接口包括注册、登录、获取用户信息与更新用户信息，用户认证模块的典型接口规格设计如表 3-1 所示。

表 3-1 用户认证模块接口设计表

接口名	请求路径	请求方法	主要参数	返回值
Register	/api/auth/register	POST	username,password	user id
Login	/api/auth/login	POST	username,password	token,user id
GetProfile	/api/user/profile	GET	无（JWT 认证）	用户信息
UpdateProfile	/api/user/update	POST	username,password	更新后信息

在登录流程里，系统利用对用户密码进行哈希校验的方式来进行身份认证。身份验证完成之后，系统会利用 HMAC-SHA256 算法来生成 JWT 令牌。令牌里包括用户标识和过期时间，服务端解析该令牌就可以判断用户是否合法登录，这样能够避免传统会话机制造成的服务端存储开销。

系统前端登录认证流程图如图 3-3 所示，其中后端处理子流程如图 3-4 所示。其中用户输入的用户信息数据流向主要由客户端发送 HTTP 请求，Web 与 Android 端分别由 Axios 与 Retrofit 与服务端通信，经由后端服务解析 Json 数据，查询数据库 MariaDB 持久化的数据，通过对比数据差异，返回不同的 HTTP Json 响应数据到客户端。若密码哈希校验通过，响应数据中负载还会包含通过 JwtUtil 生成的新令牌。

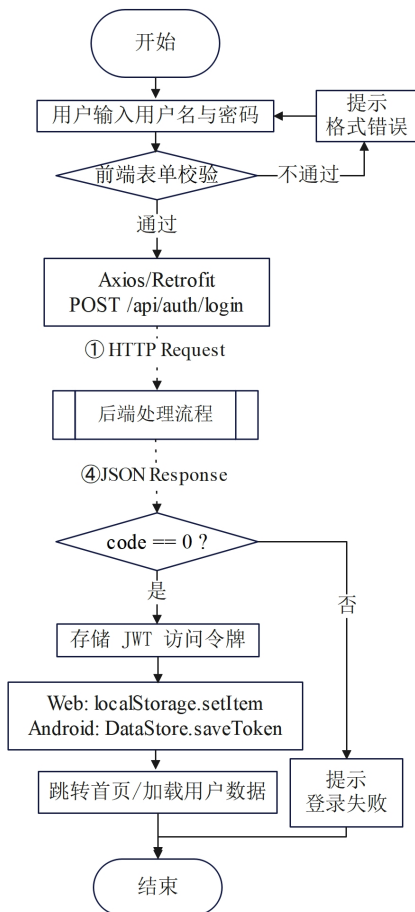


图 3-3 用户登录前端流程

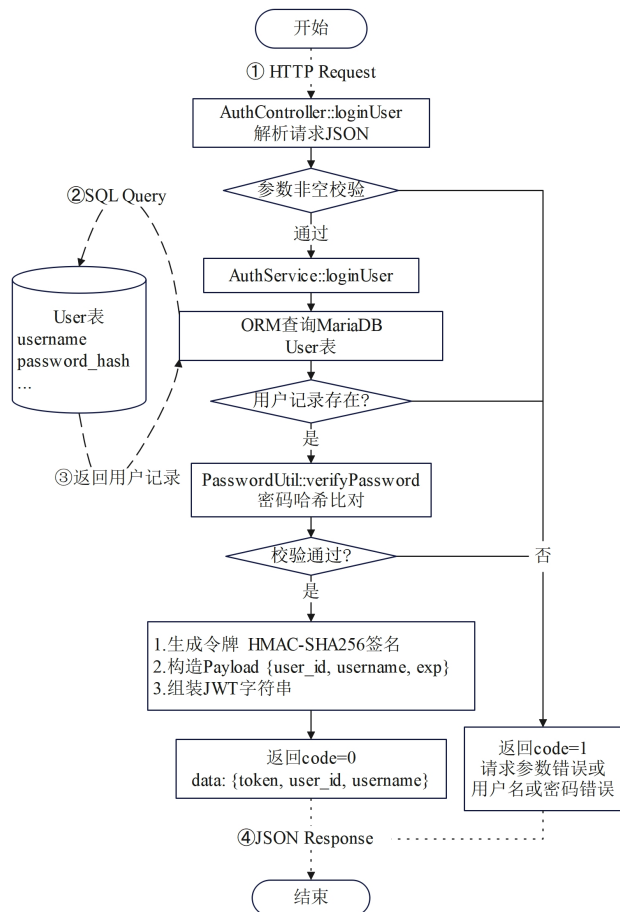


图 3-4 用户登录后端流程

3.2.2 笔记管理模块

笔记管理模块为系统核心模块，主要实现笔记的增删改查及交互操作。数据持久化采用关系型数据库，结合 Elasticsearch 以实现全文检索。该模块典型接口如表 3-2 所示。

表 3-2 笔记管理模块接口设计表

接口名	请求路径	请求方法	主要参数	返回值
CreateNote	/api/note/create	POST	title,folder_id	note_id
UpdateNote	/api/note/update	POST	note_id,content 等	状态信息
GetNoteDetail	/api/note/detail	GET	note_id	笔记详细信息
SearchNotes	/api/note/search	GET	keyword,from,size	笔记列表
GetNoteTags	/api/notes/tags	GET	note_id	标签列表
LikeNote	/api/note/like	POST	note_id	状态信息
UnlikeNote	/api/note/like	DELETE	note_id	状态信息

在实现上，系统使用了数据库和搜索引擎这两种存储机制。笔记数据先写入 MariaDB 数据库，随后利用异步任务同步到 Elasticsearch 以构建倒排索引，以此兼顾数据一致性与检索性能。同时，数据运用了软删除策略，通过标记字段来控制笔记删除状态，避免因误删造成的数据不可逆丢失。

笔记管理模块前端交互流程如图 3-5 所示，用户操作经过 Pinia 状态管理和 Axios 拦截器到服务端。服务端处理子流程如图 3-6 所示，各类 CRUD 操作依类型分流：对创建与更新，先校验所有权，再落盘并进行异步同步索引；对于删除，系统会执行软删除操作；查询则由 Elasticsearch 处理，利用 IK 分词和高亮配置返回关联结果，同时将索引命中的前两条标签存入历史记录。点赞、收藏等交互行为由 ORM 进行入库操作，并更新统计计数。

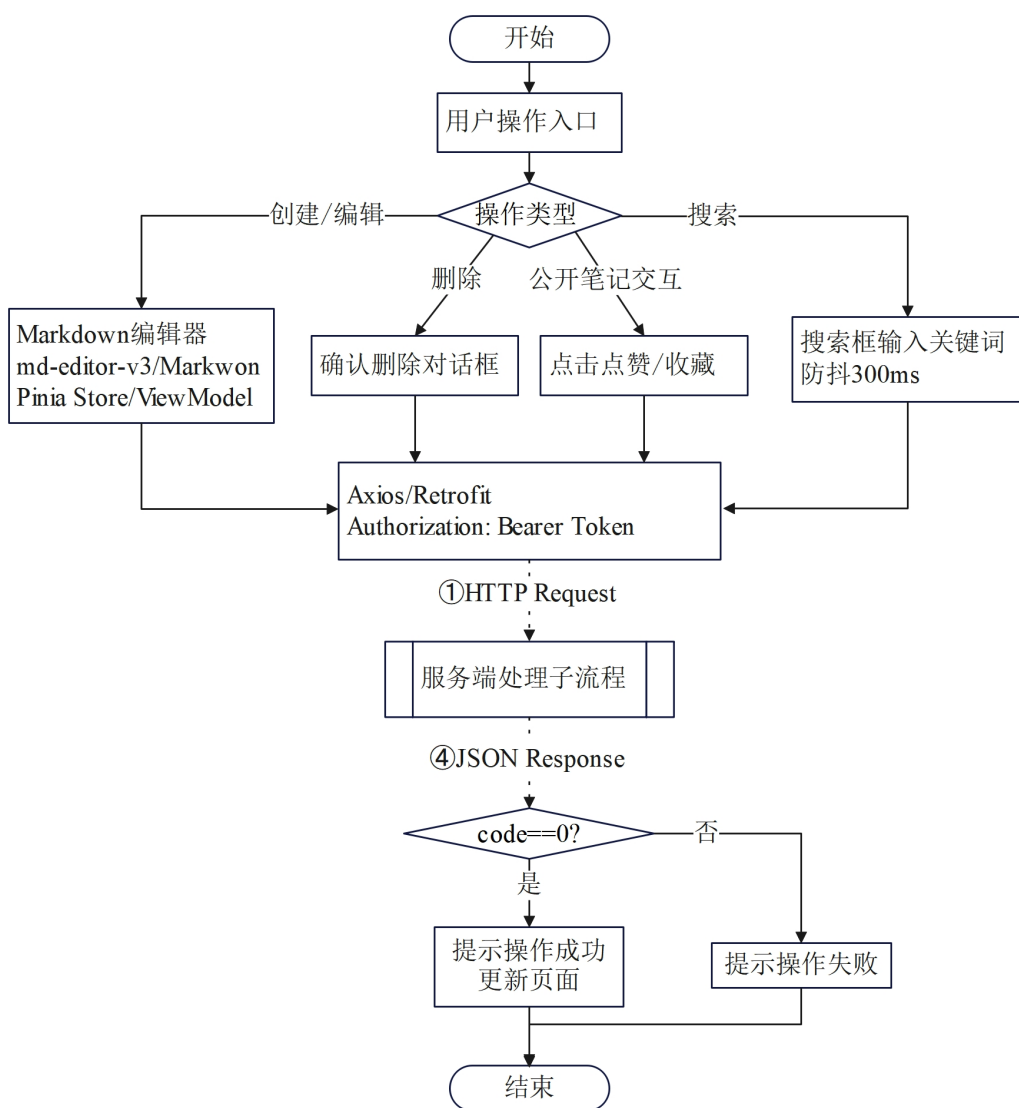


图 3-5 笔记管理前端流程

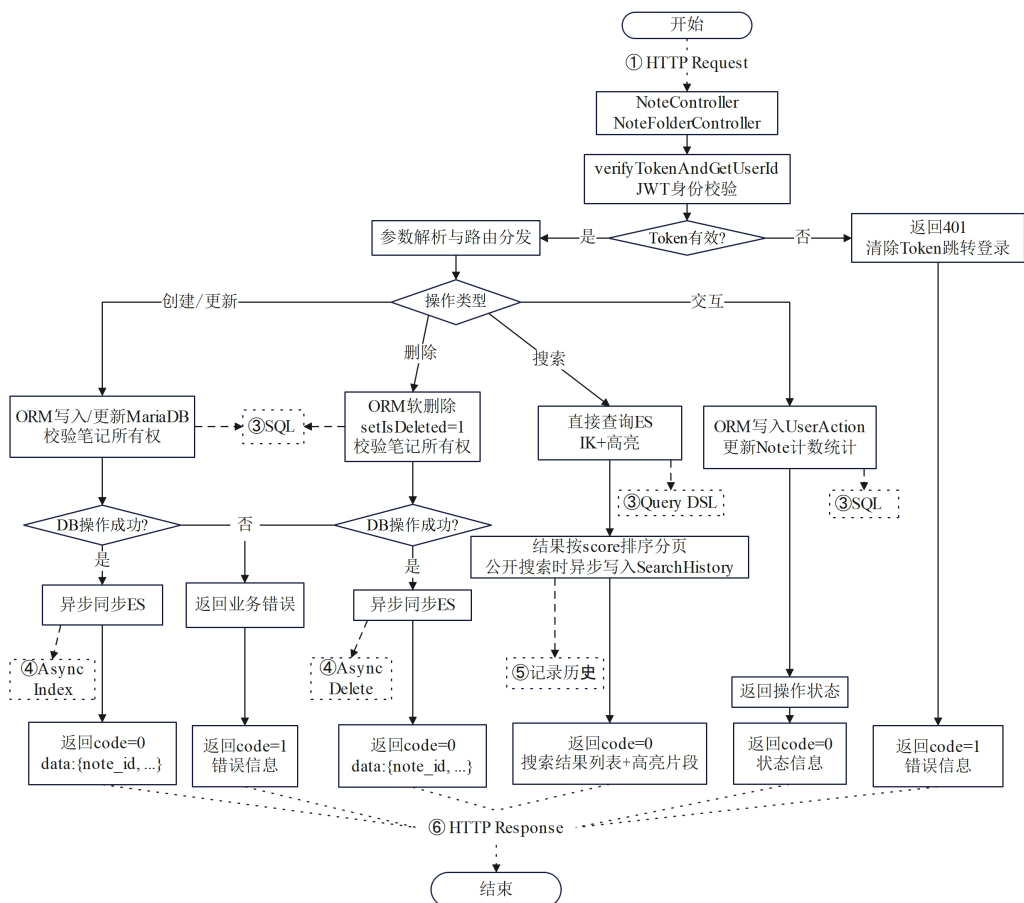


图 3-6 笔记管理后端流程

图 3-7 更直观的展示了笔记管理这一流程中数据的流向。

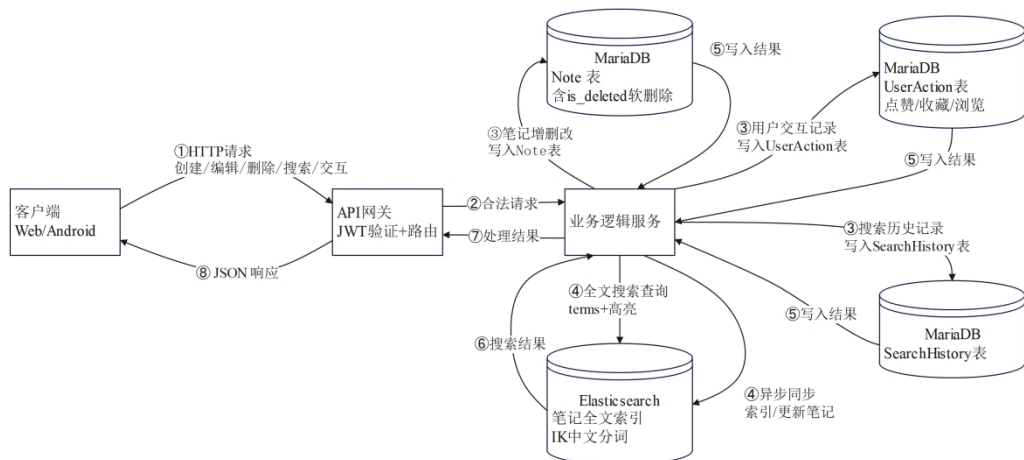


图 3-7 笔记管理数据流图

笔记创建与智能处理这一关键协作流程，其时序可描述归纳如下。第一，用户在客户端侧输入笔记标题、摘要等信息并发起创建笔记请求，使用 POST 方法将笔记数据及 JWT 令牌负载上传至服务端。第二，服务端收到请求后会开展参数的核查和身份验证，若验证失败未通过则返回 40x 错误码。第三，若服务端验证通过，系统将笔记实体写入数据库，待持久化完成之后即刻发送成功的响应标识。第四，系统同时启动异步任务：把笔记内容同步至 Elasticsearch 中用于构建索引，从而实现后续的全文检索能力；同时用户可选择调用 AI 服务对内容进行语义分析并生成相应的标签，最后将该标签写回笔记记录。图 3-8 给出笔记创建与智能处理的具体时序。

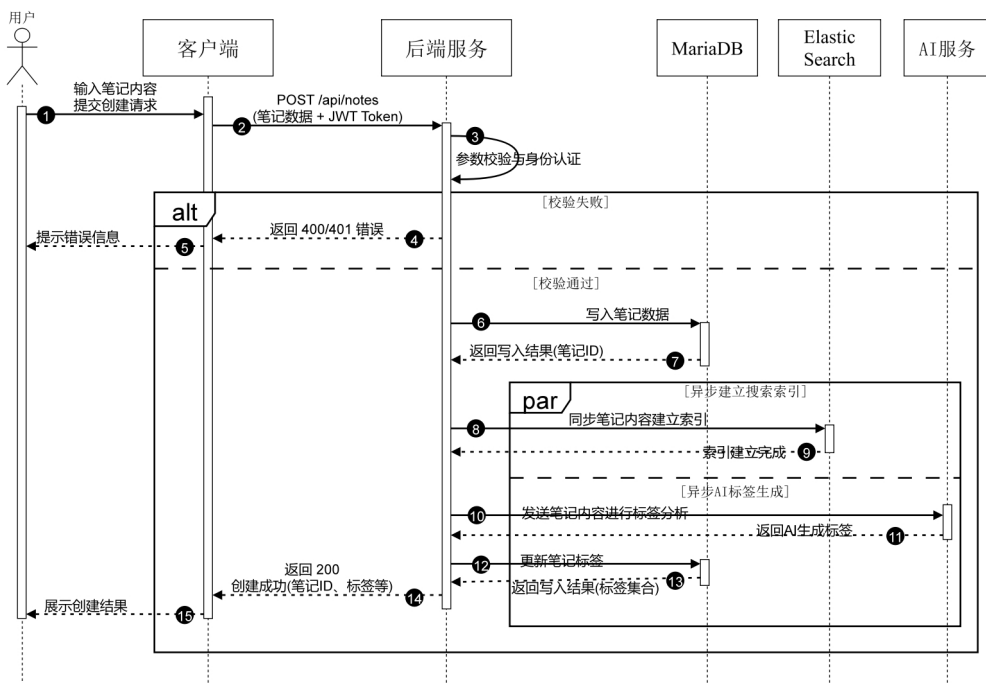


图 3-8 笔记创建时序图

3.2.3 文件夹管理模块

文件夹管理模块实现笔记的层次化组织，支持多级目录结构。用户可以对文件夹进行创建、查询及删除操作，并将笔记归类至指定目录中。文件夹管理模块的对外接口详见表 3-3 所示。

表 3-3 文件夹管理模块接口设计表

接口名	请求路径	请求方法	主要参数	返回值
CreateFolder	/api/folder/create	POST	name,parent_id	folder_id
ListFolders	/api/folder/list	GET	parent_id	文件夹列表
DeleteFolder	/api/folder/delete	POST	folder_id	状态信息
UpdateFolder	/api/folder/update	POST	folder_id,name	状态信息

该模块通过自关联结构构建树形目录，并在删除操作中采用级联处理策略，保证目录结构的闭环一致性。同时，通过内建循环检测机制避免非法层级关系的产生。

3.2.4 文件与 OCR 模块

此模块负责文件上传及图像文本识别两项机能，是系统支持多模态数据的关键组件。该模块接口设计见表 3-4。

表 3-4 文件与 OCR 模块接口设计表

接口名	请求路径	请求方法	主要参数	返回值
UploadFile	/api/file/upload	POST	file,note_id	file_id
ListFiles	/api/file/list	GET	user_id	文件列表
DeleteFile	/api/file/delete	POST	file_id	状态信息
GetFileStatus	/api/file/status	GET	file_id	文件状态
GetFileInfo	/api/file/info	GET	file_id	文件信息
OcrRecognize	/api/ocr/recognize	POST	file	file_id
GetStatus	/api/ocr/status	GET	file_id	OCR 状态

文件存储依托 MinIO 对象存储，采取异步上载过程。对于 OCR 识别，系统调用外部服务，模型为 PaddleOCR-VL-1.5。对图片或文档实施解析，解析出的文本内容自动存储为新笔记。整体处理遵循“先响应、后处理”的异步模式以提升系统响应速度。其完整处理流程如图 3-9 所示。

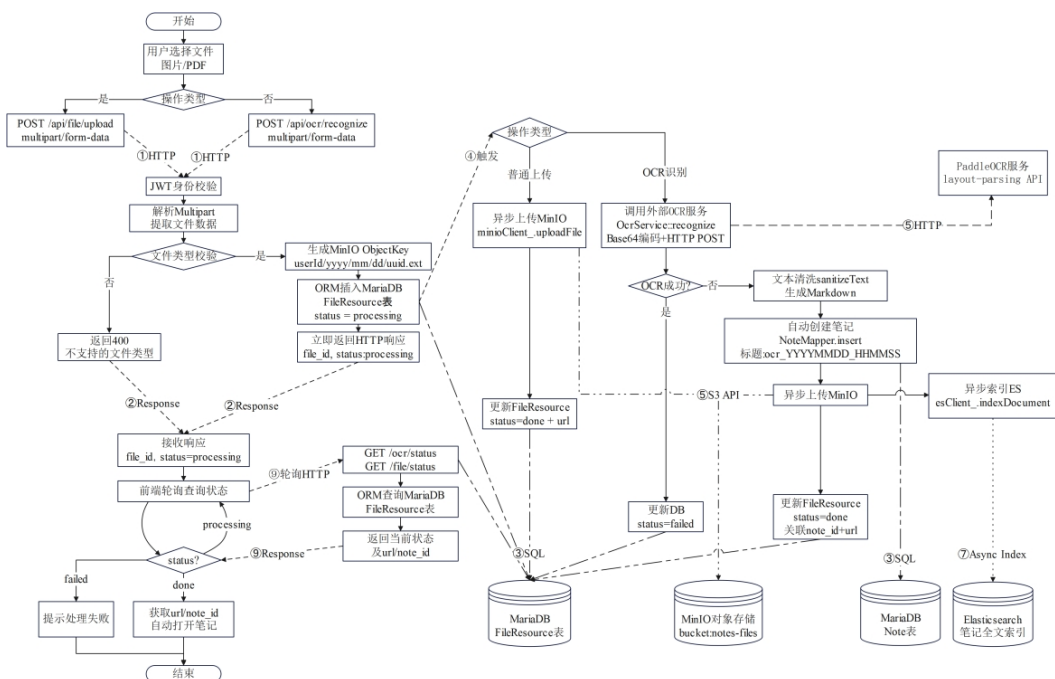


图 3-9 OCR 处理流程

OCR 识别处理的典型时序可凝练为：用户选定图片或文档并发起识别请求，客户端以 `multipart/form-data` 形式上传文件。服务端首先鉴别身份与文件类型，校验通过后将文件转存至 MinIO 并记录待处理状态，同时向客户端返回任务标识。此后异步调用 OCR 服务执行文本辨识，待识别结果返回将更新任务状态为完成，并通过识别文本创建新笔记。客户端可通过轮询接口查询处理进展，全程时序如图 3-10 所示。

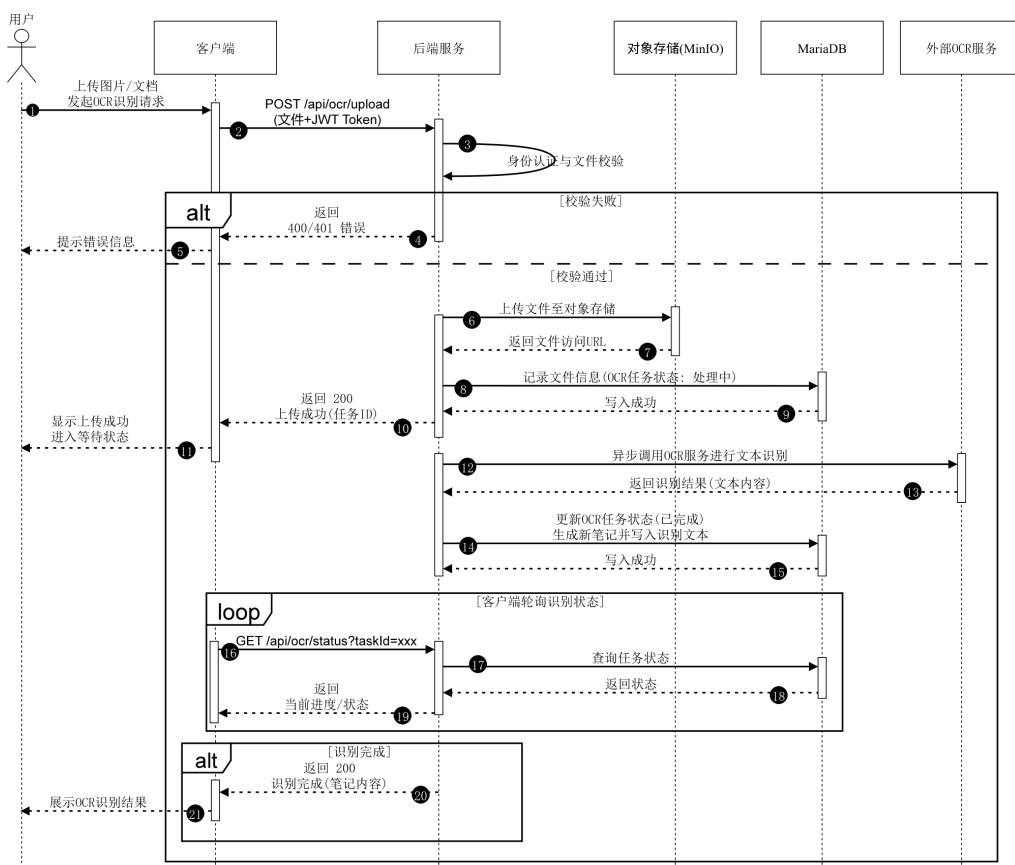


图 3-10 OCR 识别时序图

3.2.5 标签管理模块

标签管理模块的主要目的是实现笔记内容的语义分类及管理。考虑到标签同义词以及语义层级问题，系统摒弃了基础的标签管理功能。如 `Cpp`、`cpp` 和 `C++` 属于同义词，而且全都属于编程语言这一大标签。用户能够重新生成笔记标签，系统不再提供用户自定义标签和标签的增删改功能。系统采用固定标签池的方式记录标签，然后通过 LLM 进行标签自动生成。考虑到标签池数量较多，容易造成较长上下文，实际选取上下文大的模型，例如 `kimi-k2.6` 或 `deepseek` 模型。标签管理模块对外暴露的接口设计如表 3-5 所示。

表 3-5 标签管理模块接口设计表

接口名	请求路径	请求方法	主要参数	返回值
GenerateNoteTagsByAi	/api/notes/tags/ai	POST	note_id	标签列表
GetHotTags	/api/tags/hot	GET	无	标签列表

该模块通过分析笔记内容，并利用预定义的标签集合，自动产生和笔记内容相关联的标签，从而提高了笔记的结构化程度。同时，系统利用了 Elasticsearch 对标签的使用频率进行聚合统计，形成热门标签，给用户提供参考，为系统推荐提供基础。

3.2.6 搜索模块

搜索模块利用 Elasticsearch 来建立全文检索引擎，该模块是系统提高数据访问效率的重要组成部分。系统支持对笔记标题、正文、标签等多个字段进行复合查询，同时也具备关键词高亮的功能。搜索功能依靠笔记管理模块内的 SearchNotes 接口来完成，具体的设计情况如表 3-2 所示。通过运用倒排索引进行快速检索，结合相关度评分机制来获得最优匹配结果，从而明显提高查找效率和结果匹配率。

全文搜索的交互时序如下。用户在客户端输入关键词并且点击搜索，通过 GET 方式携带 JWT 把请求发送给服务端。服务端进行鉴权和校验参数操作，然后把关键词传递给 Elasticsearch。ES 引擎会对分词、相关度评分、高亮配置逐个进行执行，并返回原始数据集、评分、高亮片段。服务端对结果进行整理，并按照用户标识过滤和重构高亮字段，最后客户端把返回的笔记列表、高亮内容、相应相关度评分渲染。该过程如图 3-11 所示。

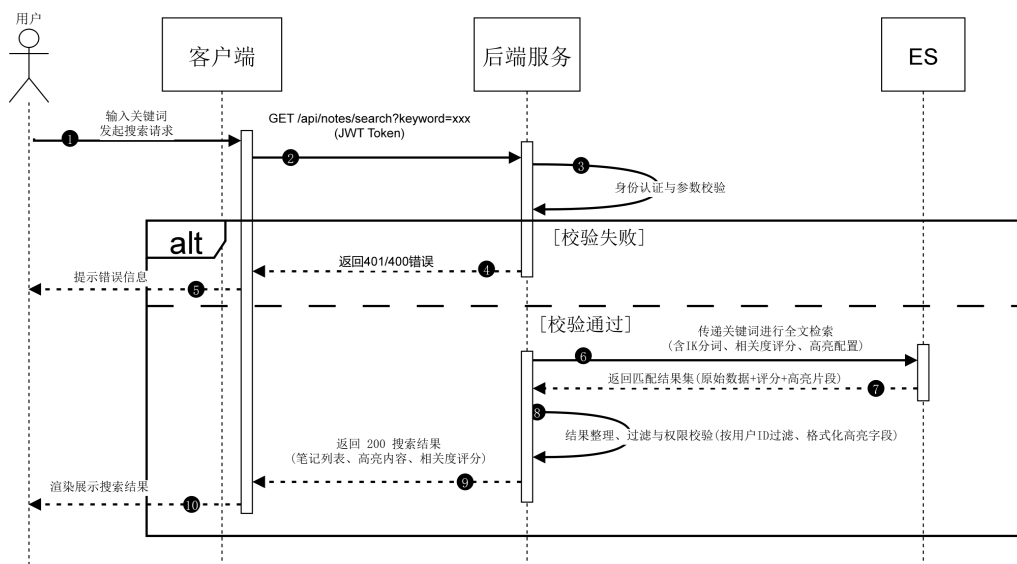


图 3-11 全文搜索时序图

3.2.7 推荐模块

推荐模块根据用户行为数据提供个性化内容推荐，是系统智能化的重要体现。该模块通过 GET 方法请求 `/api/recommend/notes` 接口，主要参数为 `page`、`page_size`，响应为笔记列表，后端接口名为 `RecommendNotes`。

推荐模块利用用户行为数据来提供个性化内容推荐，体现了系统的智能化。该模块使用 GET 方法进行请求 `/api/recommend/notes` 接口，其中主要参数包括 `page` 和 `page_size`，响应内容是笔记列表。

推荐策略依靠标签和行为共同进行过滤。因新用户行为数据较稀疏，系统使用热门标签和搜索历史来形成初始推荐内容。针对老用户，系统提取浏览、点赞和收藏记录来形成兴趣模型，并运用时间衰减函数来确定偏好权重，进而生成个性化推荐列表。若候选标签稀少，系统会使用兜底策略，这样能确保推荐服务的稳定。推荐系统的时序图如图 3-12 所示。

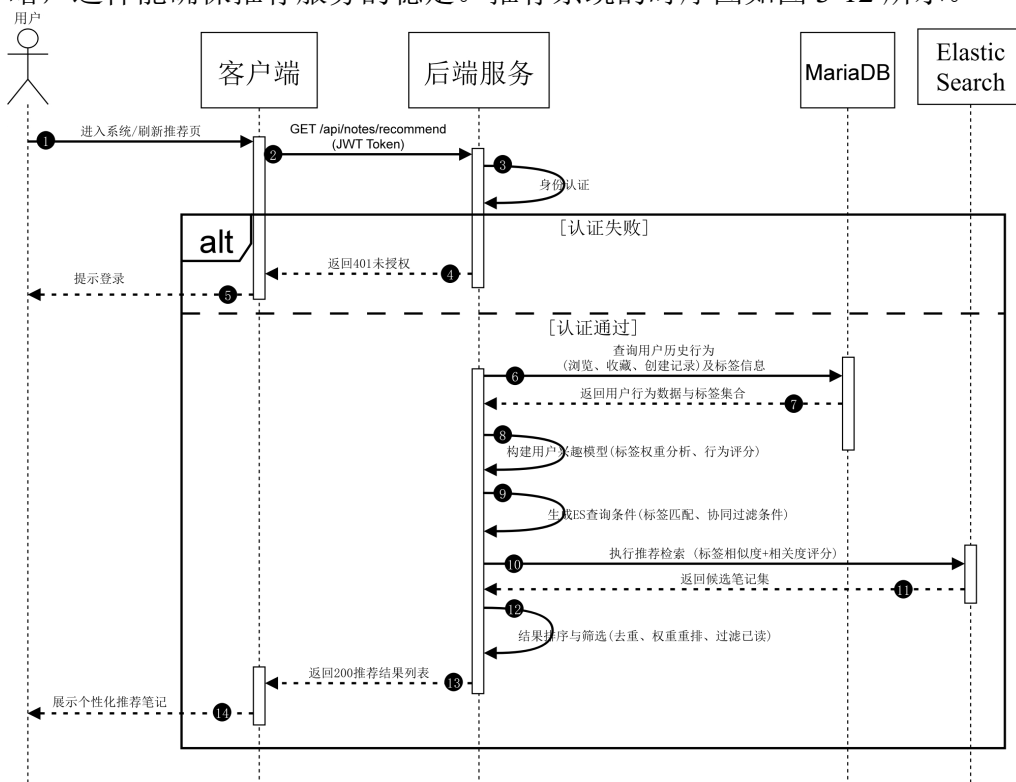


图 3-12 推荐系统时序图

推荐系统时序可抽象为：用户进入推荐页面或刷新时，客户端发送推荐请求。后端认证后查询用户近 30 日行为记录与标签集合，若行为数充沛则构建用户兴趣模型并生成 ES 查询条件，执行标签相似度与相关度检索；否则转向冷启动策略。结果经去重、权重重排与已读过滤后，封装为推荐列表返回客户端展示。

图 3-13 是推荐模块的流程。用户点击获取推荐后，后端首先判断近 30 天行为数：少于 20 将启用冷启动算法；否则通过兴趣分、搜索记录和热门标签等计算推荐标签。合并标签后检索 ES，若无结果则返回最新相关的公开笔记，最后返回列表。

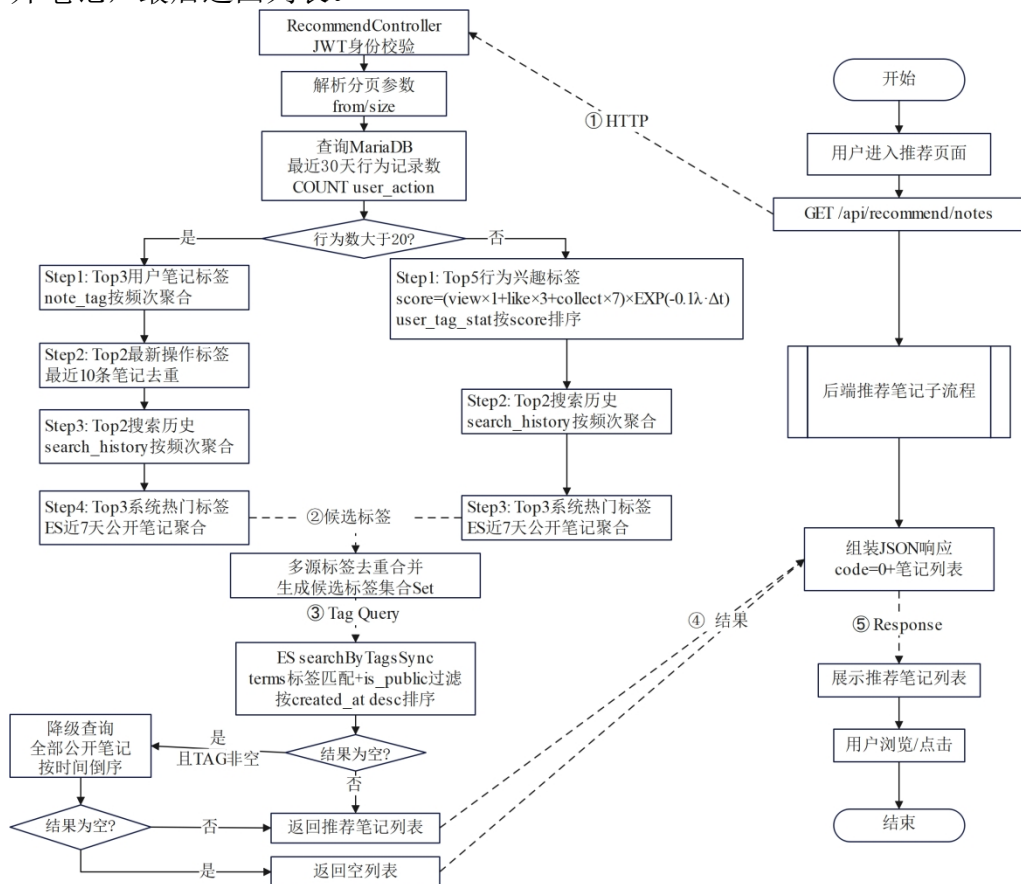


图 3-13 推荐系统流程

3.3 数据库设计

数据库设计是系统实现的重要基础，其合理性直接影响系统的性能、扩展性及数据一致性。本节结合系统功能需求，采用关系型数据库 MariaDB 进行数据存储，并通过合理的表结构设计实现数据的高效组织与管理。

3.3.1 数据库概念模型设计

根据系统实际需求，本文设计的智能笔记管理系统主要包含用户、笔记、文件夹、标签、文件资源等多个实体。各实体之间通过关联关系共同构成系统的数据模型。

其中，用户实体与笔记实体之间存在一对多关系；笔记与标签之间为

多对多关系；文件夹与笔记之间为一对多关系；用户与标签统计之间为一对多关系。系统总体 E-R 图如图 3-14 所示。

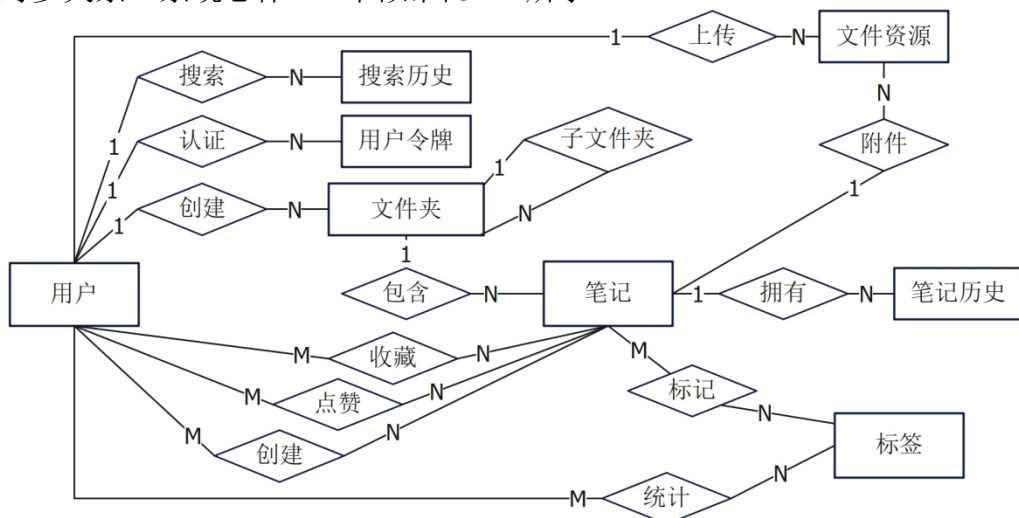


图 3-14 系统总体 E-R 图

用户实体用于存储系统用户的基本信息，是系统身份认证与权限控制的基础。其主要属性包括用户 ID、用户名、密码、邮箱及注册时间等。用户实体模型图如图 3-15 所示。

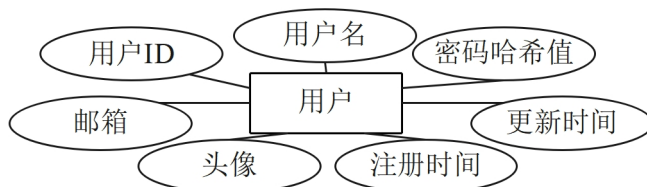


图 3-15 用户实体模型图

笔记实体是系统的核心数据实体，用于存储用户创建的笔记内容。其主要属性包括笔记 ID、所属用户 ID、标题、内容、摘要、公开状态及创建时间等。笔记实体模型图如图 3-16 所示。

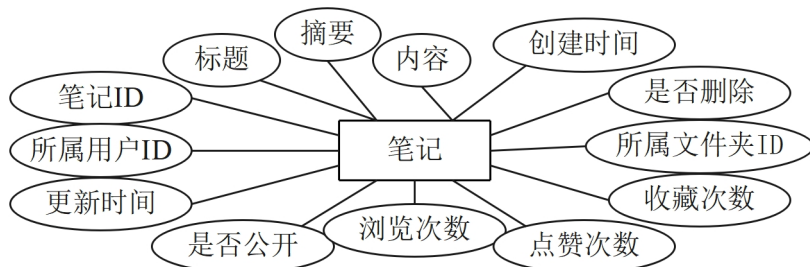


图 3-16 笔记实体模型图

文件夹实体用于组织笔记的层级结构。其主要属性包括文件夹 ID、所

属用户 ID、名称及父文件夹 ID，通过自关联关系实现树形结构。文件夹实体模型图如图 3-17 所示。

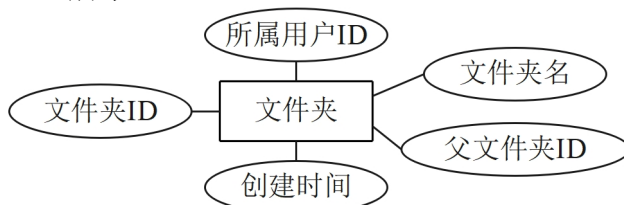


图 3-17 文件夹实体模型图

标签实体用于描述笔记的语义分类信息。其主要属性包括标签 ID 及标签名称。标签与笔记之间通过关联表实现多对多关系。标签实体模型图如图 3-18 所示。

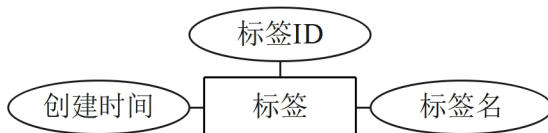


图 3-18 标签实体模型图

文件资源实体用于记录用户上传的文件信息。其主要属性包括文件 ID、用户 ID、文件名、存储路径、访问 URL 及处理状态等。图 3-19 为文件资源实体模型图。

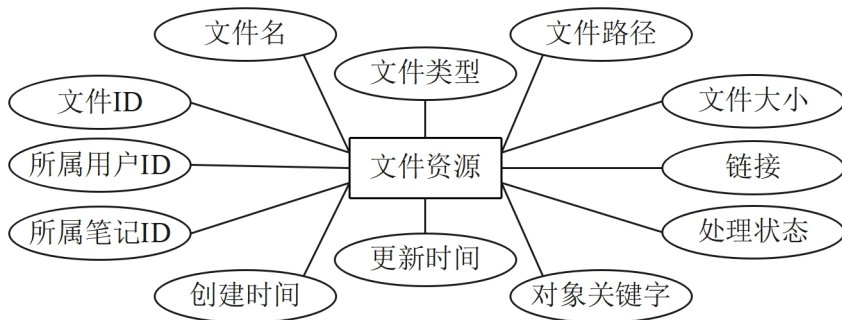


图 3-19 文件资源实体模型图

用户登录令牌实体用于记录用户登录会话的令牌信息，保障系统访问安全。其主要属性包括令牌 ID、用户 ID、令牌字符串、过期时间及创建时间等。图 3-20 为用户登录令牌实体模型图。

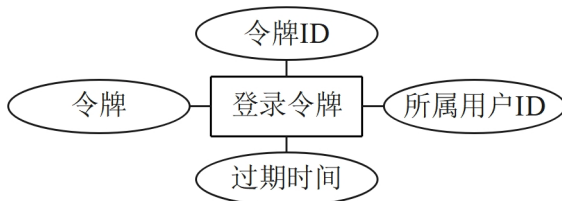


图 3-20 用户登录令牌实体模型图

搜索历史实体用于记录用户的笔记搜索行为数据，为数据统计与个性化推荐提供支撑。其主要属性包括搜索 ID、用户 ID、搜索关键词及创建时间等。图 3-21 为搜索历史实体模型图。

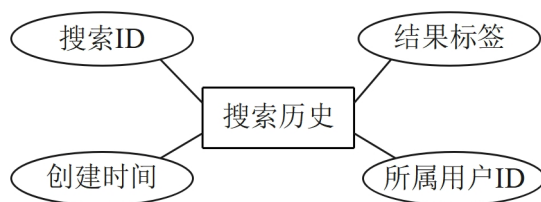


图 3-21 搜索历史实体模型图

笔记历史实体用于记录笔记修改的历史版本数据，支持笔记版本回溯与内容恢复。其主要属性包括历史 ID、笔记 ID、历史内容、版本号及创建时间等。图 3-22 为笔记历史实体模型图。

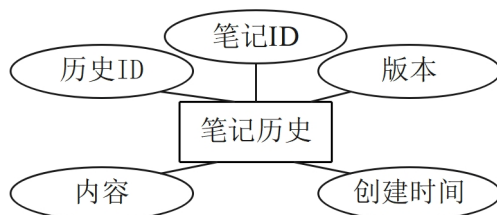


图 3-22 笔记历史实体模型图

3.3.2 数据库物理模型设计

在概念模型设计的基础上，本小节进一步给出数据库的物理结构设计。以下选取系统中核心数据表进行说明。

用户表用于存储用户基本信息，其结构设计如表 3-6 所示。

表 3-6 用户表 (user)

字段名	类型	允许空值	备注
id	BIGINT	否	用户 ID，主键
username	VARCHAR(50)	否	用户名，唯一
password_hash	VARCHAR(255)	否	密码哈希值
email	VARCHAR(100)	是	邮箱
created_at	DATETIME	否	创建时间
updated_at	DATETIME	否	更新时间
avatar	VARCHAR(255)	是	头像链接

用户标签统计表用于记录用户行为数据，其结构设计如表 3-7 所示。此表主要目的是优化服务端推荐模块：若每次获取推荐都进行数据库的 join 与 count，会提高数据库的负载。采用聚合表能减少用户行为表的查询，是推荐模块的重要组成部分。

表 3-7 用户标签统计表 (user_tag_stat)

字段名	类型	允许空值	备注
user_id	BIGINT	否	用户 ID, 联合主键
tag_id	BIGINT	否	标签 ID, 联合主键
view_count	INT	否	浏览量
like_count	INT	否	点赞量
collect_count	INT	否	收藏量
last_action_time	DATETIME	否	最后修改时间

笔记表用于存储用户的笔记内容, 其结构设计如表 3-8 所示。主要字段为笔记 ID, 用户 ID, 标题, 是否公开等。

表 3-8 笔记表 (note)

字段名	类型	允许空值	备注
id	BIGINT	否	笔记 ID, 主键
user_id	BIGINT	否	用户 ID, 外键
title	VARCHAR(255)	否	标题
content	TEXT	否	内容
summary	VARCHAR(255)	是	摘要
folder_id	BIGINT	是	所属文件夹 ID, 外键
is_public	TINYINT	否	是否公开
is_deleted	TINYINT	是	是否删除
created_at	DATETIME	否	创建时间
updated_at	DATETIME	否	更新时间
view_count	INT	是	浏览量
like_count	INT	是	点赞量
collect_count	INT	是	收藏量

文件夹表用于实现笔记的分类管理, 其结构设计如表 3-9 所示。用户在删除文件夹时, 后端服务会删除子文件夹以及文件夹中的笔记。

表 3-9 文件夹表 (folder)

字段名	类型	允许空值	备注
id	BIGINT	否	文件夹 ID, 主键
user_id	BIGINT	否	所属用户 ID, 外键
name	VARCHAR(100)	否	文件夹名
parent_id	BIGINT	是	父文件夹 ID, 外键
created_at	DATETIME	否	创建时间

用户行为表用于统一记录用户对笔记的各类操作行为, 其结构设计如表 3-10 所示。其中 action_type 为枚举类型, view 取值为 1, like 取值为 2, collect 取值为 3。

表 3-10 用户行为表 (user_action)

字段名	类型	允许空值	备注
id	BIGINT	否	行为 ID, 主键
user_id	BIGINT	否	用户 ID, 外键
note_id	BIGINT	否	笔记 ID, 外键
action_type	TINYINT	否	行为类型
created_at	DATETIME	否	行为时间

标签表用于存储系统中的标签信息, 其结构设计如表 3-11 所示。由于系统预制标签池, 理论上该表数据不会被修改。

表 3-11 标签表 (tag)

字段名	类型	允许空值	备注
id	BIGINT	否	标签 ID, 主键
name	VARCHAR(50)	否	标签名, 唯一
created_at	DATETIME	否	创建时间

笔记标签关联表用于实现笔记与标签的多对多关系, 其结构设计如表 3-12 所示。文件资源表用于存储上传文件的信息, 表 3-13 是其结构设计。

表 3-12 笔记标签关联表 (note_tag)

字段名	类型	允许空值	备注
note_id	BIGINT	否	笔记 ID, 联合主键
tag_id	BIGINT	否	标签 ID, 联合主键

表 3-13 文件资源表 (file_resource)

字段名	类型	允许空值	备注
id	BIGINT	否	文件 ID, 主键
user_id	BIGINT	否	所属用户 ID, 外键
note_id	BIGINT	是	所属笔记 ID, 外键
file_name	VARCHAR(255)	否	文件名
file_path	VARCHAR(255)	否	文件路径
file_type	VARCHAR(255)	否	文件类型
file_size	BIGINT	是	文件大小
object_key	VARCHAR(255)	否	对象键值
url	VARCHAR(255)	是	访问 url
status	VARCHAR(20)	否	处理状态
created_at	DATETIME	否	创建时间
updated_at	DATETIME	否	更新时间

笔记点赞表与收藏表用于记录用户的笔记点赞与收藏行为, 其中点赞表结构设计如表 3-14 所示, 收藏表与点赞表结构相同。

表 3-14 笔记点赞表 (note_like)

字段名	类型	允许空值	备注
user_id	BIGINT	否	用户 ID, 联合主键
note_id	BIGINT	否	被点赞笔记 ID, 外键
created_at	DATETIME	否	创建时间

搜索历史表用于记录用户的搜索行为记录, 其结构设计如表 3-15 所示。

表 3-15 搜索历史表 (search_history)

字段名	类型	允许空值	备注
id	BIGINT	否	历史记录 ID, 主键
user_id	BIGINT	否	所属用户 ID, 外键
query	VARCHAR(255)	是	搜索结果标签 ID 集合
created_at	DATETIME	否	创建时间

用户令牌表用于存储用户登录认证令牌信息, 结构设计如表 3-16 所示。

表 3-16 用户令牌表 (user_token)

字段名	类型	允许空值	备注
id	BIGINT	否	令牌 ID, 主键
user_id	BIGINT	否	所属用户 ID, 外键
token	VARCHAR(255)	否	令牌
expired_at	DATETIME	是	失效时间
created_at	DATETIME	否	创建时间

笔记历史表用于记录笔记历史, 其结构设计如表 3-17 所示。

表 3-17 笔记历史表 (note_history)

字段名	类型	允许空值	备注
id	BIGINT	否	历史 ID, 主键
note_id	BIGINT	否	笔记 ID, 外键
content	TEXT	是	内容
version	INT	是	版本
created_at	DATETIME	否	创建时间

3.4 本章小结

本章完成了系统整体的设计工作。系统采用前后端分离架构, Web 端和 Android 端通过 RESTful API 对后端服务进行调用。数据层整合 MariaDB、Elasticsearch 和 MinIO, 对结构化数据和非结构化数据进行处理。功能上包括了笔记管理、全文搜索和个性化推荐这些模块。同时利用时序图和 E-R 图确定了主要流程和数据库结构, 为后面实奠定基础。

第 4 章 系统实现与测试

4.1 核心算法与机制实现

本文以智能笔记管理为目标，在推荐、搜索、OCR 处理和认证等重要环节上，设计并实现了若干算法和机制。本文用户行为和标签的推荐算法作为中心，并且依靠全文检索算法的优化方案和异步处理机制，共同支撑系统的智能化水平和运行效果。

4.1.1 基于用户行为与标签的推荐算法

推荐模块将分层兴趣建模和标签匹配检索进行结合，来实现个性化推荐的目标。算法主要依据用户行为数据的丰富情况选择不同的推荐方式，以此兼顾冷启动问题和推荐精度问题。

系统会根据用户在最近 30 天内所留下的行为记录数量，包括浏览、点赞和收藏等方面，来对用户进行分层。若行为数据的数量少于 20 条，系统就会将该用户判定为新用户，否则视作老用户。该划分方法可以将兴趣信息稀疏和兴趣信息充分这两种用户场景进行区分。

系统根据老用户的浏览记录，采用行为加权和衰减的方法来计算兴趣标签。用户行为分成浏览、点赞和收藏这三种类型，并且分别给它们设定不等的权重，收藏的权重最大，点赞的权重其次，浏览的权重最小。具体某标签的兴趣得分计算公式如式 4-1 所示。

$$Score_{tag} = \sum_{i \in \{view, like, collect\}} (w_i \times count_i) \times e^{-\lambda \cdot \Delta t} \quad (4-1)$$

式 4-1 中 $w_{view} = 1$ ， $w_{like} = 3$ ， $w_{collect} = 7$ ， $\lambda = 0.1$ ， $count_i$ 表示该标签的行为数量。 λ 表示时间衰减系数， Δt 表示用户最近一次行为到当前时间的间隔天数。代码利用 SQL 进行直接聚合计算，省去了逐条行为遍历所造成的性能开销。算法运用指数衰减函数，使近期行为对推荐结果的影响变大，从而提高推荐的时效性。系统按照兴趣得分对标签进行排序，并且选择前 5 个权重较高的标签，作为用户兴趣标签集合。

针对新用户，由于缺乏充分的行为数据，系统运用多源信息融合的冷启动方法，从用户现有的笔记标签、最新的操作标签、搜索历史和系统热门标签里筛选出候选标签，再进行去重合并，形成初始兴趣集合。具体而言，系统选取用户前三笔记高频标签、前二最近笔记新标签、前二搜索历史标签以及前三全局热门标签作为用户兴趣标签集合。

系统在对兴趣标签进行建模后，利用 Elasticsearch 进行标签匹配查询，在公开笔记库里查找含有相关标签的内容，并按照相关程度排序形成推荐列表。为保证系统稳定性，本文设计多级兜底模式。当标签匹配结果为空时，系统会自动进行降级处理，并且返回最新公开的笔记，以此来保证推荐结果可用性。推荐算法的主要逻辑代码如下所示。

```
vector<Note> recommendNotes(int64_t user_id, int page, int page_size) {
    int from = (page - 1) * page_size;
    int action_count = query(
        "SELECT COUNT(*) FROM user_action
        WHERE user_id = ?
        AND created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)",
        user_id);
    bool is_new = (action_count < 20);
    set<string> tags;
    if (is_new) { // 新用户
        tags.insert(queryTags(
            "SELECT t.name, COUNT(*) AS cnt FROM note_tag nt
            JOIN note n ON nt.note_id=n.id JOIN tag t ON nt.tag_id=t.id
            WHERE n.user_id=?
            GROUP BY t.name ORDER BY cnt DESC LIMIT 3",
            user_id));
        int added = 0;
        for (auto& t : queryTags(
            "SELECT t.name FROM note n
            JOIN note_tag nt ON n.id=nt.note_id
            JOIN tag t ON nt.tag_id=t.id WHERE n.user_id=?
            ORDER BY n.created_at DESC LIMIT 10",
            user_id))
            if (tags.insert(t).second && ++added >= 2) break;
        tags.insert(queryTags(
            "SELECT query FROM search_history WHERE user_id=?
            GROUP BY query ORDER BY COUNT(*) DESC LIMIT 2",
            user_id));
        tags.insert(esGetHotTags(3));
    } else { // 老用户
        tags.insert(queryTags(
            "SELECT t.name FROM user_tag_stat uts JOIN tag t
            ON uts.tag_id=t.id WHERE uts.user_id=? ORDER BY
            (uts.view_count*1+uts.like_count*3+uts.collect_count*7)*
            EXP(-0.1*COALESCE(
                DATEDIFF(NOW(),uts.last_action_time),30))
            DESC LIMIT 5",
```

```

        user_id));
tags.insert(queryTags(
    "SELECT query FROM search_history WHERE user_id=?
    GROUP BY query ORDER BY COUNT(*) DESC LIMIT 2",
    user_id));
tags.insert(esGetHotTags(3));
}
auto res = esSearchByTags(tags, from, page_size);
if (res.empty() && !tags.empty())
    res = esSearchByTags({}, from, page_size);
return res;
}

```

4.1.2 ES 全文检索实现

系统在搜索功能实现时，运用基于倒排索引的全文检索方法，提高检索速度和匹配准确度。在 Elasticsearch 中建立 notes 索引，对笔记的标题、正文和摘要字段进行了分词处理。中文分词使用了 IK 分词器，以实现中文语义级别的检索。本模块基于 C++ 编写 ES 客户端，利用 HTTP 协议同 ES 服务端进行交互，以实现文档的处理工作。

系统在进行查询时使用了布尔查询和多字段匹配策略，并对多个文本字段进行加权检索。其中标题字段权重最高，以突出其语义重要性。正文权重相对较低。以 Elasticsearch 返回的相关性评分 Score 为排序依据，实现按照内容匹配程度进行排序机制。

此外，系统支持关键词高亮，在返回结果里对命中词进行标记，提高用户的可读性体验。如果正文部分没有包括高亮内容，摘要字段将作为展示内容，保证界面的展示效果。

Elasticsearch 的 notes 索引映射如下。

```

"mappings": {
  "properties": {
    "content": {
      "type": "text",
      "analyzer": "ik_max_word"
    },
    "created_at": { "type": "date" },
    "is_public": { "type": "boolean" },
    "summary": {
      "type": "text",
      "analyzer": "ik_max_word"
    },
    "tags": { "type": "keyword" },
    "title": {

```

```
"type": "text",  
  "analyzer": "ik_max_word"  
},  
"updated_at": { "type": "date" },  
"user_id": { "type": "long" }  
}}
```

其中 `content` 字段为笔记正文，属于文本类型，分析器为 `IK`，`title` 为笔记的标题，而 `tags` 作为标签则为关键词类型，不会进行分词处理，整个字符串会当作一个完整的单元，进行精准匹配。

4.1.3 系统其他机制

针对用户上传的图片或 PDF 文件，本文设计并实现了一个利用 OCR 技术来提取文本、自动生成笔记的流程。该流程运用了异步任务投递和状态驱动结合的协作方式。用户在完成文件上传操作后，系统首先会在数据库里创建一条文件记录，并把该记录的状态标记为“处理中”。然后，启动一个单独的后台线程，使用 `PaddleOCR-VL-1.5` 模型来进行文本识别。系统在得到识别结果后，会对原始文本进行清洗，清洗工作包括移除多余的空白、纠正错误的字符，并且进行格式转换。然后会自动生成相应的笔记内容。该设计的主要流程采用异步方式进行执行，后台线程负责进行耗时较长的 OCR 工作，主线程不需要进行阻塞等待。状态字段和前端轮询方式一起工作，把处理进度传递给用户。异步化设计能够减轻请求队列拥堵的压力，并且使系统整体的吞吐能力得到提高。

在系统实现过程中，大部分操作都包括数据库访问、搜索引擎调用、文件上传这类 IO 密集型场景。为使性能得到提高，本文运用 `Drogon` 框架的非阻塞 I/O 模式，形成了依靠事件循环驱动的异步处理方式。具体来看，数据库查询和 HTTP 请求均被组织为回调函数形式来进行执行。针对 OCR 识别和长时间文件上传这类任务，通过单独的线程池来执行。任务完成后，利用事件循环回调主线程，以此来更新相应的业务状态。在数据一致性上，系统运用数据库优先和搜索引擎异步同步的方法。系统将全部主要业务数据同步写入 `MariaDB`，使主数据保持强一致性。接着利用异步任务把数据同步至 `Elasticsearch` 索引里。该设计会出现搜索索引短暂的滞后窗口。时间通常处于毫秒至秒之间，但工程实践可以得到更高的写入吞吐量，该方法是处理性能和一致性冲突的一种折衷方案。

用户身份认证过程采用 `JWT` 这种无状态方式进行处理。用户在完成登录操作后，服务端会生成包括用户标识和过期时间的令牌，并且使用 `HMAC-SHA256` 算法来对令牌进行签名。在接下来的请求中，客户端只需将令牌置于 HTTP 请求头即可。服务端依次对令牌结构进行解析，核验签名真伪，并校验有效期限。系统在所有验证通过后会从令牌中提取用户身

份信息，并通过该信息来执行后续的业务逻辑。在某个步骤未成功的情况下，后端会发送未经授权的响应。并发请求数量的增加，无状态设计降低了服务端在内存或分布式缓存上的占用，该设计有利于系统进行水平扩展。

4.2 系统实现效果

系统以智能笔记管理为中心业务，在 Web 端和 Android 端均完成了用户的完整交互功能。集成全文搜索、OCR 识别和个性化推荐等智能能力。Web 端使用三栏式布局，左侧是文件夹树和笔记列表，中间是 Markdown 编辑器及预览区，右侧则是笔记属性和文件管理面板。Android 端使用 Jetpack Navigation 单 Activity 多 Fragment 架构，可以进行离线编辑和后台同步。系统整体界面设计较简洁，交互流程较清晰，满足用户在多终端场景下的使用需求。下文展示主要功能页面和实现效果。

4.2.1 Web 端实现效果

系统登录页面主要负责实现用户身份认证功能，用户访问系统前端网址将默认进入该页面。页面使用卡片式布局，支持深色和浅色主题切换，主要表单实现效果如图 4-1 所示。用户输入用户名与密码后，若用户名或密码过短会提示用户注意账号安全性。



图 4-1 系统登录页面

用户注册页面主要实现新用户账户创建功能，用户可通过默认的登录页面点击立即注册跳转到该页面。该页面包括用户名唯一性校验、密码一致性校验及邮箱可选绑定等。同样对于用户名或密码过短会提示用户注意账号安全性，若用户输入的邮箱格式不正确也会提示用户存在格式问题。系统在前端与后端均进行数据合法性验证，确保注册信息的有效性，主要表单实现效果如图 4-2 所示。



图 4-2 用户注册页面

Web 端三栏式主界面是系统的主要工作区域，实现效果如图 4-3 所示。该页面整体使用左中右三栏布局。左侧边栏把工具栏、文件夹树形结构和用户信息集成在一起，中间区域显示了笔记列表。右侧边栏包括笔记属性面板和文件列表面板，并且支持在这些面板之间进行切换。左侧工具栏包括 OCR 识别入口，用户上传图片或 PDF 后系统会自动提取文本并生成笔记。右侧属性面板还包括了 AI 标签生成功能，可以按照当前笔记内容自动推荐标签，并且同步更新到 Elasticsearch 中。



图 4-3 Web 端三栏式主界面布局

笔记创建和编辑页面属于系统中心功能页面，主要进行 Markdown 格式笔记的编辑、实时预览。笔记编辑界面的实现效果如图 4-4 所示。Web 端笔记编辑界面使用双栏编辑器模式，该模式可以进行语法高亮显示和图片上传。编辑器通过 md-editor-v3 来实现，并且支持亮色和暗色主题的自适应。该页面利用 Vue3 组合式 API 进行开发，并依靠状态管理库对笔记编辑

数据进行统一维护。页面包括了顶部操作区和编辑内容区。顶部拥有笔记标题编辑、页面返回、手动保存、保存状态展示这些功能。编辑器对全套工具栏进行了自定义配置，并且支持了文本样式、代码块、表格、公式和流程图等常用 Markdown 语法的编辑。



图 4-4 笔记编辑页面

页面适配了系统主题模式，通过对编辑器底层 CSS 变量进行重写，实现了界面色彩、边框、滚动条和弹窗样式的整体适配。系统利用后端文件上传接口和状态轮询方式进行异步批量处理图片上传，以此保证文件上传的稳定和可靠。编辑过程可以实时标记笔记的未保存状态，并利用组件自定义事件进行向上交互，从而完成页面跳转和数据保存的业务逻辑，这明显提高笔记编辑的规范性、交互体验。

系统在编辑流程中加入笔记历史界面，用来增强版本追溯功能。实现效果如图 4-5 所示。界面以弹窗的形式显示，弹窗通过模态叠加层设置在编辑区上方，排列历史版本条目，分别标记当前编辑版本和各留存版本的固化时间戳。界面内设有“保存当前版本为历史”和“清空历史”这两个功能控件，针对每一条历史版本都提供了一个可以恢复该版本的操作入口。用户可以在持续编写的过程中随时调用该界面，查看笔记内容的变化，进行版本快照留存、历史记录清除或指定版本回溯。将版本维度的控制权直接交给用户，降低误改引起数据丢失的风险。



图 4-5 笔记历史界面

笔记列表主要展示用户笔记内容，包括标题、摘要和更新时间等信息，用户点击列表项会进入笔记详情页面，进行内容编辑和标签生成等相关操作。本文使用 Vue3 组合式 API 进行组件开发，利用列表循环渲染实现了单条笔记卡片的批量展示，通过组件属性传参将笔记数据和选中标识绑定在一起，并且利用自定义事件把列表点击交互行为向上传递。笔记列表界面实现的效果如图 4-6 所示。

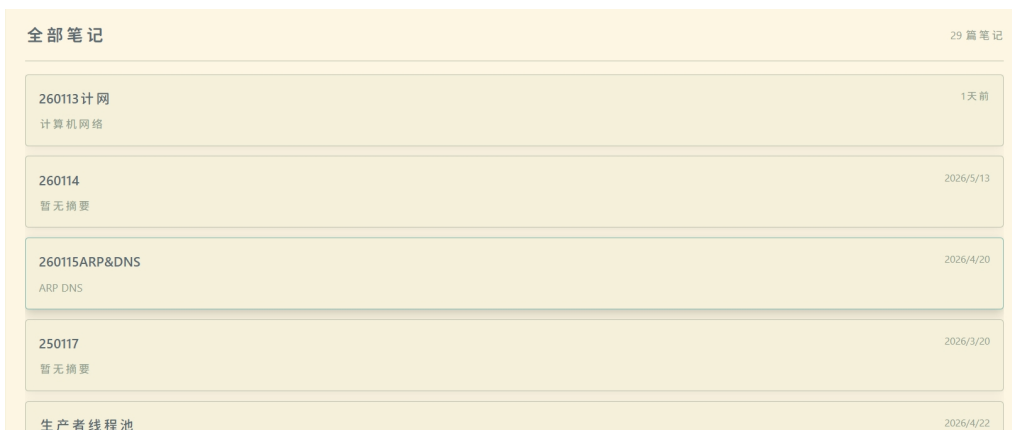


图 4-6 笔记列表页面

公开笔记浏览页面主要用于查看其他用户公开的笔记内容。页面顶部显示笔记标题与返回按钮，右侧提供点赞与收藏交互按钮；内容区采用 Markdown 预览模式渲染，支持代码高亮与表格展示，效果如图 4-7 所示。



图 4-7 公开笔记浏览页面

全文搜索页面主要实现了使用 Elasticsearch 进行关键词检索的功能。前端利用 Axios 把关键词、分页参数和搜索范围构造成 JSON，向后端发送 GET 请求，后端返回 ES 查询结果。页面支持用户输入关键词、高亮显示、结果分页及切换个人和公开范围等功能，用户可以快速返回相关笔记列表。在系统主界面上端点击搜索按钮，会弹出搜索框，效果如图 4-8 所示。在该界面中用户可以选取搜索个人的笔记或者搜索其他人的公开笔记。用户在进行关键词输入操作时系统会进行持续 300 毫秒的防抖处理，这样做是为了防止用户在打字过程中频繁触发请求。



图 4-8 系统搜索框弹窗界面

搜索结果界面效果如图 4-9 所示，该界面对用户的搜索结果进行了高亮匹配。若搜索的是公开笔记，页面会显示公开笔记的作者部分信息。高亮实现逻辑：后端 ES 在检索时自动给匹配到的关键词加<mark>标签，前端配

合 CSS 设置背景即可达到高亮效果。

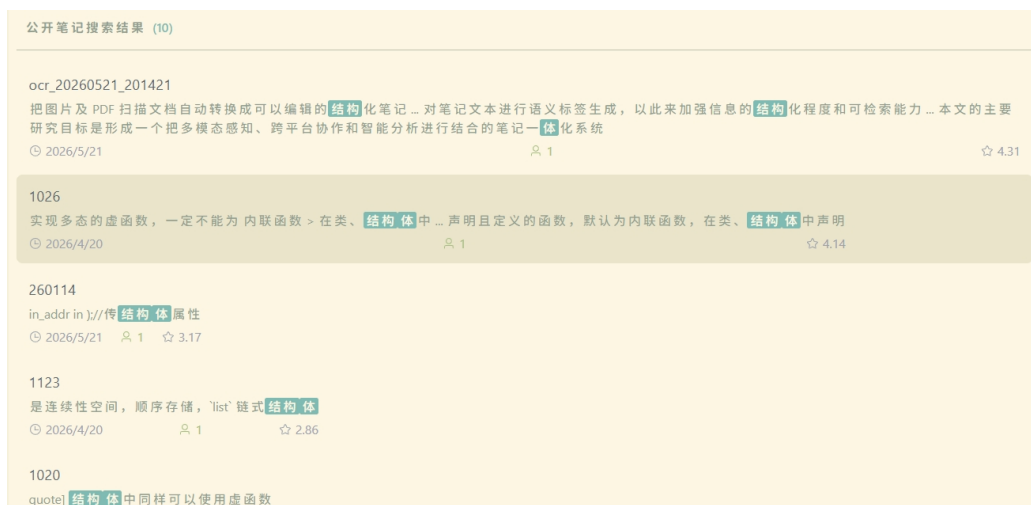


图 4-9 笔记搜索结果页面

搜索过程中，系统为笔记各个字段设置不同权重，其中标题权重最高，摘要次之，笔记内容的权重最低。标题字段的 **boost** 值最高，ES 计算分数时标题匹配的笔记排序会更靠前；前端拿到结果后，匹配的笔记卡片会显示 ES 计算的 **score** 分数，即相关度得分，保留两位小数展示在卡片右下角。

文件夹管理页面主要实现笔记的分类管理功能，支持文件夹的创建、重命名、删除及层级嵌套展示。用户可通过树形结构直观管理笔记组织结构，实现效果如图 4-10 所示。文件夹中包含子菜单按钮，通过子菜单，用户可在该文件夹下创建笔记或子文件夹，重命名或删除文件夹。



图 4-10 文件夹管理页面

数据概览界面以统计卡片与简明图表相结合的形式，凝聚式呈现用户创作活动的量化轮廓。实现效果如图 4-11 所示。



图 4-11 笔记数据概览界面

笔记数据概览界面首行排布四类指标卡：笔记总篇数、文件夹总数、累积字数及本月新建笔记数量；中部区域绘制近七日创建趋势折线图，辅以文件夹维度的笔记分布饼图；底部罗列最近编辑的笔记条目。页面通过多维数据的聚合陈列，赋予使用者对个人知识资产累积态势的概观性把握。右上角嵌入“返回首页”按钮，维持导航路径的连贯。

推荐页面主要展示系统根据用户兴趣模型生成的个性化内容。页面采用列表布局展示推荐笔记，随着用户行为积累，推荐结果逐渐趋于个性化；新用户采用冷启动策略，综合笔记标签、搜索历史与全局热门标签进行推荐，实现效果如图 4-12 所示。

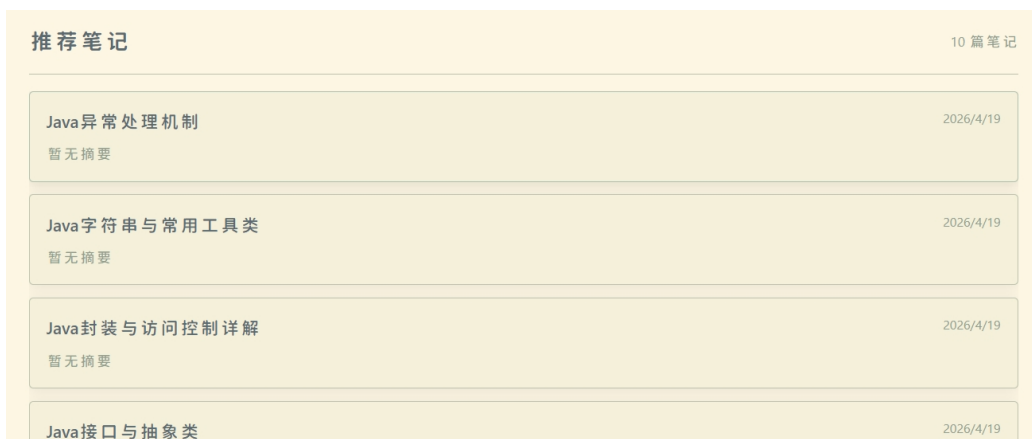


图 4-12 推荐页面

文件上传与资源管理页面主要实现用户文件的上传、状态查看及访问链接管理。系统通过状态字段（处理中/完成/失败）展示文件处理进度，支持异步任务的可视化管理与 MinIO 对象存储对接，效果如图 4-13 所示。

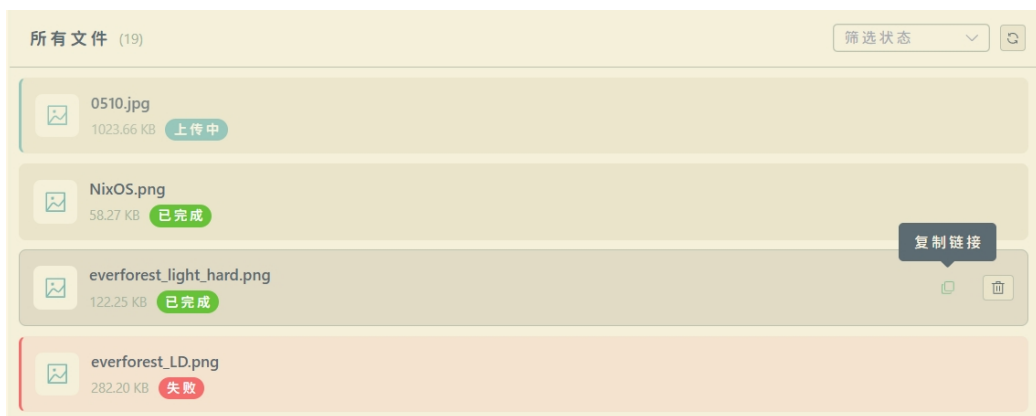


图 4-13 文件管理页面

个人中心界面含有身份鉴权信息，系统表现层有偏好管理功能。实现效果如图 4-14 所示。该界面可通过主界面左下角用户信息部分子菜单点击进入。



图 4-14 个人中心界面

个人中心界面划分成四个区块。基本信息区展示头像、用户名和绑定邮箱。账号安全区提供密码变更表单，该表单包括旧密和新密输入项。外观设置区实现了深色和浅色模式的即时更换，默认为 Everforest 配色。并

且开放主题色选择器，让用户按照个体感知偏好来订制界面基调。区罗列了系统采用的技术栈构成和开源许可协议信息，全局右上角固定放置了返回首页按钮。该界面整合了身份、安全和个性化这三个方面，保持访问路径简洁的同时，也让操作预期达成一致。

回收站界面运用卡片统计和操作列表联动的方式，给用户提供了对已删除笔记进行暂存管理、二次处理的功能，实现效果如图 4-15 所示。界面顶部的两个卡片区域分别显示已经删除笔记的数量和即将自动清理的倒计时提示。界面下面放置了全部恢复和清空回收站这两个批量操作按钮。主体列表把已删除笔记的标题和删除时间等摘要信息按顺序列出，每条记录的末尾设有恢复和彻底删除的快捷入口。该“总览—选取—批量或单项执行”的交互模式，既有效防止误删数据，也让使用者获得了颗粒化的控制权限。



图 4-15 回收站界面

时间轴界面以纵贯式时序流线，对笔记的创建轨迹按时间呈现。界面右上角为“返回首页”按钮，保持导航链路一致。主视觉为一条自上而下延展的垂直轴杆，依日期倒叙逐层锚定节点，每个节点承载该日新增笔记的聚合卡片。卡片内有笔记标题、创建时刻、内容摘要及所属文件夹，由线条与色标的层层析分，不同日期的笔记群清晰界隔。通过按照时间线的排布，用户的创作规律与知识沉淀过程得以具象化，达成时序脉络与空间布局的双向链接。时间轴界面实现效果如图 4-16 所示。



图 4-16 时间轴界面

4.2.2 Android 端实现效果

Android 端登录页面主要实现了移动端用户身份认证，实现效果如图 4-17 所示。界面采用 Material Design 风格，支持用户名、密码输入及登录状态保持。页面利用 LoginFragment 实现，使用 EditText 表单录入用户名和密码。Fragment 绑定控件，点击登录时输入框的内容将传递到 ViewModel 处理。使用 MVVM 架构传输数据，因此 Fragment 只涉及 UI，业务逻辑在 LoginViewModel 中进行处理。成功登录后 Token 写入 DataStore，自动跳转至主界面。通过 Retrofit 发送 POST 请求，配合 Gson 自动解析 JSON。Android 端注册界面实现效果如图 4-18 所示。

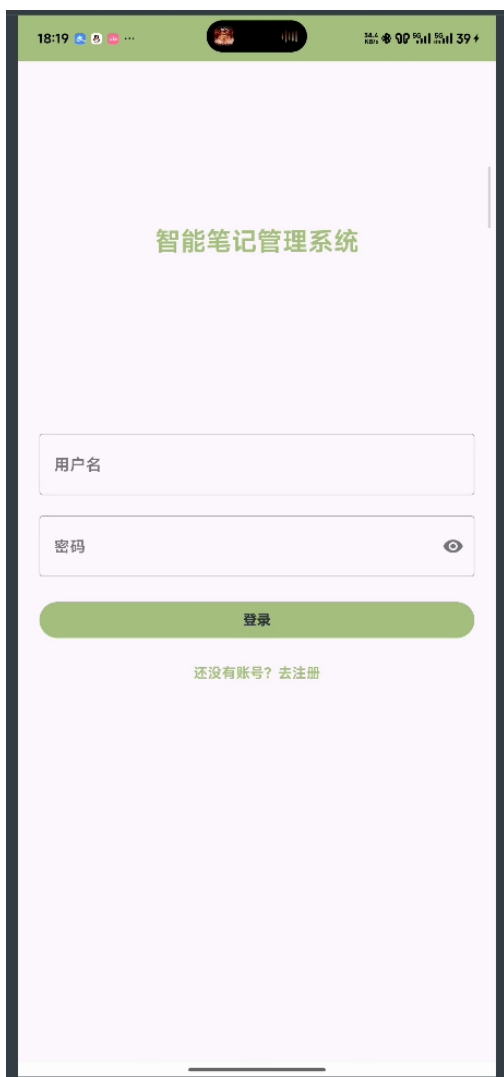


图 4-17 Android 端登录页面

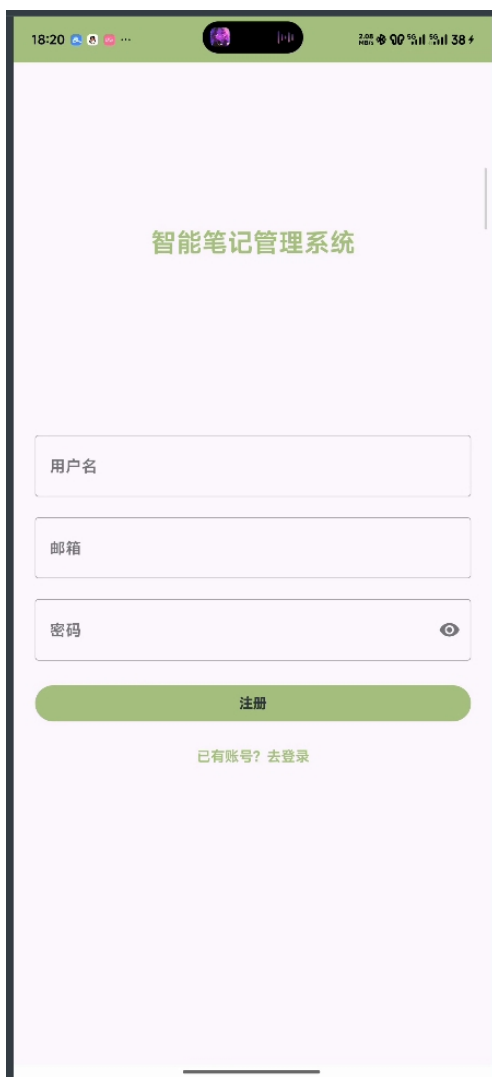


图 4-18 Android 端注册页面

Android 端笔记列表与文件夹管理页面采用 RecyclerView 与 TreeAdapter 实现树形结构展示文件夹层级与笔记条目，支持文件夹展开/收起、长按弹出 BottomSheet 文件夹操作子菜单。通过子菜单可进行新建笔记、重命名与删除操作，实现效果如图 4-19 所示。TreeAdapter 继承自 ListAdapter，内部通过 DiffUtil 刷新局部，由 FolderNode 和 NoteNode 两种节点构成。通过 level 字段控制缩进，每级加深 32dp。文件夹点击会切换 isExpanded 状态，同时箭头图标旋转 90 度。长按菜单通过 BottomSheetDialog 实现，新建笔记和重命名菜单项通过 MaterialAlertDialogBuilder 结合 EditText 实现。页面内嵌 SwipeRefreshLayout，可下拉刷新同步数据。

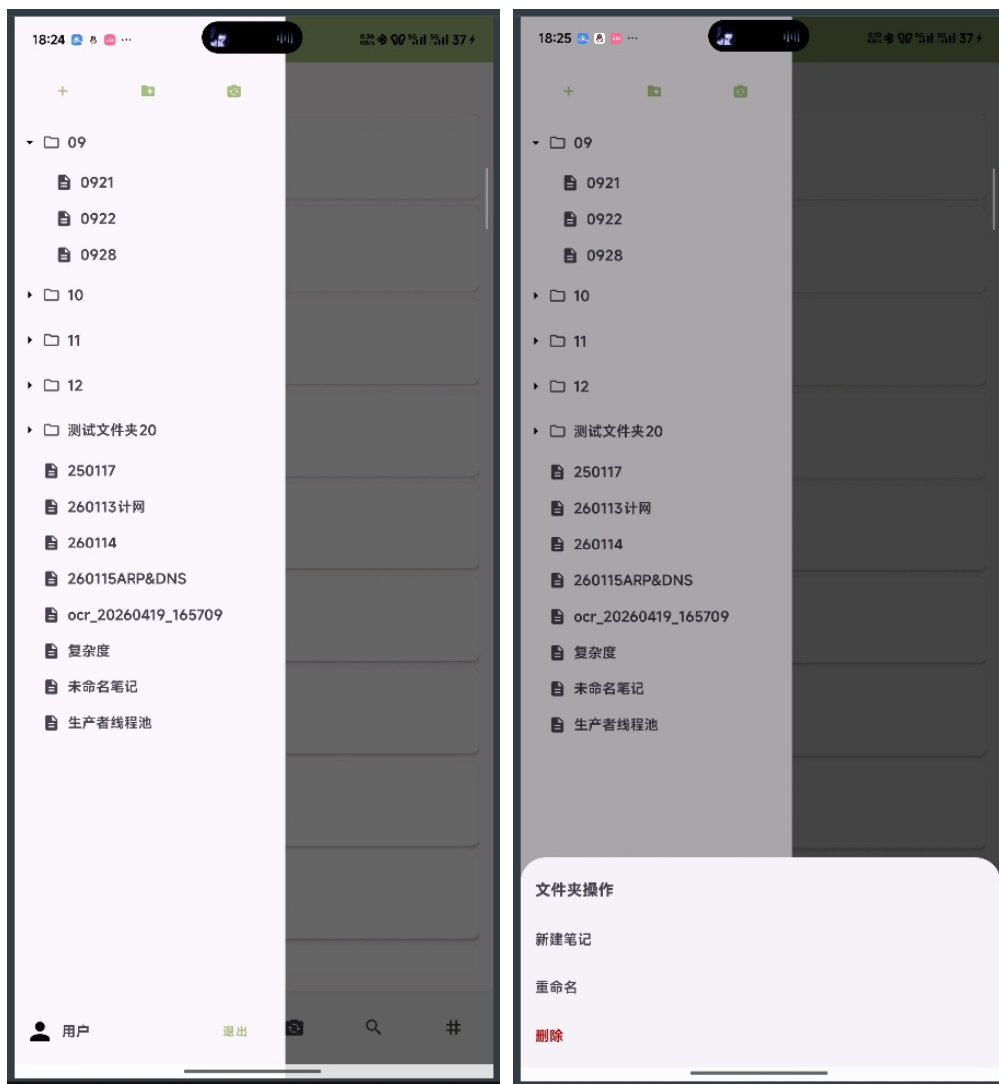


图 4-19 Android 端笔记列表与文件夹管理页面

Android 端笔记编辑与预览页面支持 Markdown 纯文本编辑与渲染预览两种模式切换。页面是 NoteEditorFragment，编辑区利用 EditText 实现，把 TextView 嵌套在 ScrollView 中实现了预览区，点击顶部按钮可切换两个 View 的可见性，实现效果如图 4-20 所示。编辑模式下运用以下方法实时自动保存：ViewModel 创建每 5 秒轮询一次的协程，若内容有变动调用 saveNote()通过 Retrofit 将更新发送到后端。同时页面使用 TextWatcher 进行监听输入。onPause 时也会更新，防止状态切换导致数据丢失。利用 Markwon 引擎渲染 Markdown 实现了预览模式，支持图片加载与表格展示。Markwon 实例还结合 CoilImagesPlugin 实现图片加载，TablePlugin 支持表格，公式和代码块也能正常显示。



图 4-20 Android 端笔记编辑与预览页面

Android 端笔记详情页面主要用于查看笔记完整信息，顶部展示标题、作者信息与标签，底部提供点赞与收藏按钮，浏览他人公开笔记时可进行互动，实现效果如图 4-21 所示。界面通过 NotePreviewFragment 实现，进入时先调用 viewModel.loadNote()获取详情，同时后台发送一个浏览请求给后端进行行为统计。标签列表通过 ChipGroup 动态实现，每个标签通过 addView 新建一个 Chip 添加到组件，没有标签则显示“暂无标签”。互动页面正文通过 Markwon 渲染，图片和表格都可正常显示，实现效果如图 4-22。点赞为 toggle 逻辑，NotePreviewViewModel 里判断当前 has_liked 状态，决定调 likeNote 还是 unlikeNote，操作完毕后再次获取详情刷新计数和按钮文案。收藏逻辑流程与点赞相同。



图 4-21 Android 端笔记详情页面



图 4-22 Android 端笔记互动页面

Android 端全文搜索页面提供关键词实时检索功能,支持个人笔记与公开笔记范围切换。搜索结果采用 RecyclerView 列表展示,标题与内容关键词高亮显示,实现效果如图 4-23 所示。界面为 SearchFragment,搜索框由 EditText 结合 TextWatcher 实现,用户输入文本时将关键词交给 SearchViewModel; ViewModel 实现了 400 毫秒防抖,用 delay(400)结束上一次的搜索协程,避免频繁请求。用户可通过顶部 Switch 控制搜索个人笔记还是公开笔记,切换后会携带当前关键词重查一次。RecyclerView 配合 ListAdapter, ViewHolder 中使用 Html.fromHtml 将后端返回的 highlight_title 和 highlight_content 转成带<mark>标签的 Spanned 显示出来。点击结果时判断 author_id,若为个人笔记转到编辑器,公开笔记则在预览页显示。

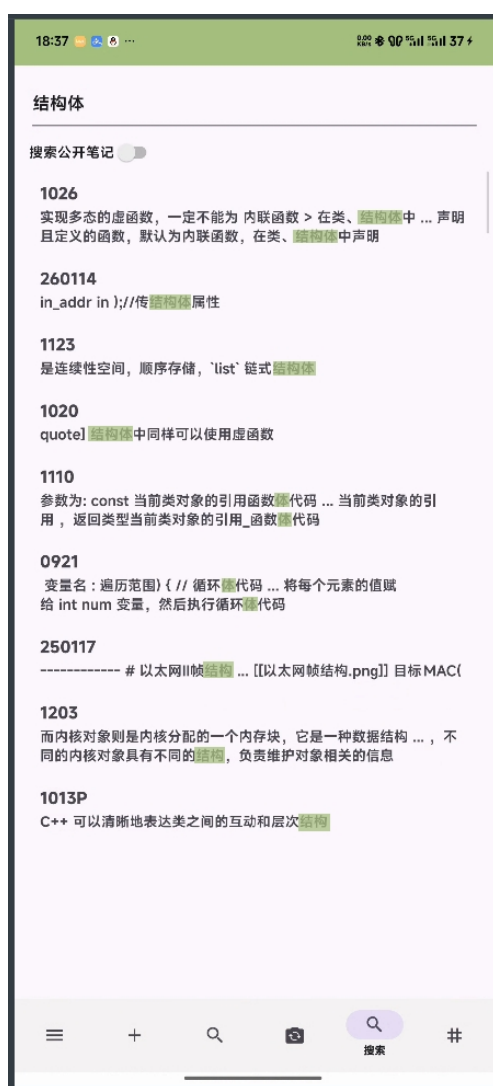


图 4-23 Android 端笔记搜索结果页面

Android 端推荐页面整合了个性化推荐与笔记辅助功能。推荐页基于用户画像返回公开笔记列表，支持下拉刷新，实现效果如图 4-24 所示；工具面板页面通过 TabLayout 切换“笔记信息”与“文件管理”两大模块，笔记信息面板支持摘要编辑、公开状态切换、标签展示、点赞/收藏及笔记删除；文件管理模块支持本地上传、链接复制与文件删除，实现效果如图 4-25。



图 4-24 Android 端推荐页面



图 4-25 Android 端文件工具面板页面

限于篇幅，后台接口交互过程、Elasticsearch 索引同步细节、MinIO 对象存储管理链路及搜索历史记录、笔记互动统计等辅助功能不再逐一列出，但均已在系统中实现并通过测试验证。

4.3 系统测试

4.3.1 单元测试

用户登录功能需要输入用户名与密码，系统对其进行校验，验证成功后生成 JWT 令牌并返回客户端；若验证失败，则返回对应错误信息。测试结果如表 4-1 所示，所有测试均已通过。

表 4-1 登录功能测试表

编号	测试内容	预期结果	实际结果
1	用户名: admin 密码: 123456	登录成功, 返回 Token	符合预期
2	用户名: admin 密码: 1234567	登录失败, 提示密码错误	符合预期
3	用户名: user101 密码: 123456	登录失败, 提示用户不存在	符合预期
4	Token 过期 访问接口	返回未授权	符合预期

用户注册功能需校验用户名唯一性及密码合法性。测试结果如表 4-2 所示。测试覆盖对用户的用户名和密码长度安全性建议，邮箱格式合法性的检测等。测试结果符合预期，所有测试均已通过。

表 4-2 用户注册测试表

编号	测试内容	预期结果	实际结果
1	新用户名: user 密码: 123456	注册成功	符合预期
2	重复用户名: user 密码: 123456	注册失败 提示用户名已存在	符合预期
3	新用户名: user01 短密码: 123	注册失败 提示密码过短	符合预期
4	短用户名: aka 密码: 123456	注册失败 提示用户名过短	符合预期
5	邮箱: u@163.com 密码: 123456	注册成功	符合预期
6	邮箱: u163.com 密码: 123456	注册失败 提示邮箱格式错误	符合预期

笔记模块为系统核心功能之一，测试包括创建、更新及权限校验。测试结果如表 4-3 所示。其中还包含用户与公开笔记交互的测试，这一部分的测试会影响到用户能否正常获取推荐。同样，所有测试均已通过。

表 4-3 笔记操作测试表

编号	测试内容	预期结果	实际结果
1	合法标题	创建成功	符合预期
2	更新笔记	更新成功	符合预期
3	修改本人笔记	修改成功	符合预期
4	删除存在笔记	删除成功	符合预期
5	删除不存在笔记	删除失败	符合预期
6	收藏他人笔记	收藏成功	符合预期
7	点赞他人笔记	点赞成功	符合预期

测试 Elasticsearch 搜索能力，包括关键词匹配与高亮。测试结果如表 4-4 所示。

表 4-4 搜索功能测试表

编号	测试内容	预期结果	实际结果
1	存在关键词	返回相关笔记	符合预期
2	不存在关键词	返回空列表	符合预期
3	中文关键词	正确分词匹配	符合预期
4	搜索公开内容	返回相关笔记	符合预期

测试文件上传与识别流程。测试结果如表 4-5 所示。

表 4-5 OCR 测试表

编号	测试内容	预期结果	实际结果
1	上传图片	进入处理状态	符合预期
2	查询状态	显示处理状态	符合预期
3	OCR 成功	生成笔记	符合预期
4	OCR 失败	状态为失败	符合预期

测试文件夹层级及操作逻辑。测试结果如表 4-6 所示。能否正常处理笔记文件夹的操作影响到用户使用体验，所有测试均已通过。

表 4-6 文件夹功能测试表

编号	测试内容	预期结果	实际结果
1	创建文件夹	创建成功	符合预期
2	更名文件夹	更新成功	符合预期
3	删除文件夹	删除成功	符合预期
4	非法层级操作	操作失败	符合预期
5	创建子文件夹	操作成功	符合预期
6	创建子笔记	创建成功	符合预期
7	删除子笔记	删除成功	符合预期

测试标签生成逻辑。测试结果如表 4-7 所示。所有测试均已通过。

表 4-7 标签功能测试表

编号	测试内容	预期结果	实际结果
1	正常笔记内容	生成标签	符合预期
2	空内容	不生成标签	符合预期

测试推荐算法效果。测试结果如表 4-8 所示。主要测试用户行为数据数量小于 20 条以及无数据时的兜底策略。经测试，新用户和老用户均能获取到相应的推荐内容，保底策略也能正常运行。

表 4-8 推荐功能测试表

编号	测试内容	预期结果	实际结果
1	新用户	返回热门内容	符合预期
2	老用户	返回个性化推荐	符合预期
3	无数据	返回兜底数据	符合预期

测试用户信息获取与修改。测试结果如表 4-9 所示。这部分测试用户个性化定制化内容。所有测试均已通过。

表 4-9 用户信息测试表

编号	测试内容	预期结果	实际结果
1	获取信息	返回用户信息	符合预期
2	修改信息	修改成功	符合预期
3	非法访问	拒绝访问	符合预期

4.3.2 集成测试

在单元测试全部通过的前提下，集成测试对各个独立的单元模块进行组合测试。这部分主要包括笔记的创建和搜索联动测试，OCR 识别生成笔记测试以及用户行为与推荐系统测试。

首先是笔记创建与搜索联动测试。测试目的是验证笔记管理模块和搜索模块协作时系统的稳定性和协同工作能力。测试流程如下：用户在客户端登录系统；然后创建一篇新笔记，输入标题与正文内容并保存；系统理论上完成了数据库写入成功后，会通过异步机制将笔记内容同步到 Elasticsearch 索引；随后用户在搜索框输入新建的笔记相关的关键词进行检索个人的笔记；系统应能够返回刚刚创建的笔记，并且前端应当正确展示高亮内容片段。该测试主要验证笔记内容的数据库写入，搜索引擎同步进而搜索展示的完整链路。经测试，笔记创建与搜索联动测试通过。

其次是 OCR 识别、笔记生成流程测试。该部分测试流程包括：用户利用 OCR 转笔记功能把图片或 PDF 文件进行上传，系统对文件进行接收并创建处理记录。系统接着在后台进行异步调用 OCR 识别服务。系统在识别完成之后会自动生成一条新的笔记，并把识别到的内容写入到笔记的正文

之中，同时把文件上传到对象存储并且更新状态。前端利用轮询方式来获取处理结果，并且把生成的笔记进行展示。该测试验证了文件上传→OCR识别→笔记生成→状态回写的完整流程。经测试，OCR识别流程测试也通过。

最后是用用户行为与推荐系统联动测试。推荐系统推荐笔记的结果涉及到用户行为数据数量。若为新用户，即新注册的账号，直接点击获取推荐按钮，系统理论上应该推送带有热门标签的笔记。当该用户通过几次搜索公开笔记后，系统理论上会为用户推荐搜索结果相关的笔记。当用户浏览以及点赞收藏若干笔记后，用户行为数据数量达到老用户标准，系统此时应当为用户推荐基于用户行为的笔记。基本流程如下：用户在系统中进行浏览、点赞及收藏操作；系统记录用户行为数据；更新用户标签兴趣统计；在推荐接口调用时，系统基于用户行为数据生成兴趣标签；利用Elasticsearch检索推荐。测试验证推荐结果是否随着用户行为发生变化，以及推荐内容是否与用户兴趣相关。经测试，新用户和老用户的推荐系统联动测试均通过。

4.4 本章小结

本章实现了智能笔记管理系统的核心功能。例如运用用户行为与标签的混合推荐算法、Elasticsearch的全文检索优化、OCR异步识别、JWT认证等关键机制。系统完成了Web端与Android端的主要页面开发。单元测试与集成测试验证了系统各功能链路的正确性与稳定性，整体运行良好。

结论

本文基于 C++ 语言的 Drogon 框架、MariaDB 数据库、Elasticsearch 以及 MinIO 等技术，设计并实现了一套智能笔记管理系统。系统集成了笔记管理、全文搜索、OCR 识别及个性化推荐等功能，解决了传统笔记系统在检索效率低、内容结构化程度不足以及个性化服务能力弱等问题。本文所作的主要工作如下。

1. 遵循软件工程的基本方法，对系统进行了整体的需求分析。包括功能需求和非功能需求。系统功能需求主要涵盖了用户认证、笔记管理、文件夹管理、标签管理、全文搜索、OCR 识别及推荐系统等模块。非功能需求主要把系统的响应性能、可扩展性、安全性和可用性等方面进行考虑。

2. 按照模块化和分层架构设计思想，进行了总体结构设计工作。系统使用了 Controller - Service - Model 分层模式。并且结合了异步非阻塞 I/O 方式，在高并发环境下达到了稳定运行。本文对主要业务流程进行了整体设计，包括笔记创建、搜索、OCR 处理和推荐流程。

3. 在详细设计阶段，本文对各个功能模块进行了接口设计和数据结构设计。确定了模块之间的交互方式和职责划分。本文重点设计了基于用户行为的推荐模型、Elasticsearch 索引结构、异步任务处理方式，提高系统整体的可维护性和扩展性。

4. 在系统实现上，本文运用 Drogon 框架进行后端服务开发。实现了用户认证 JWT、笔记管理、全文搜索和推荐算法等主要功能。本文集成了 OCR 识别服务和对象存储，实现了从文件上传到文本识别再到自动生成笔记的智能化流程。并且对系统主要功能的实现效果进行了展示和分析。

5. 在系统测试阶段，本文运用单元测试和集成测试结合的方式。对主要功能模块和中心业务流程进行验证。测试结果表明，系统在功能正确性、响应性能和模块协同方面都达到了预期目标。能够稳定地进行笔记创建、搜索和智能处理等主要操作。

虽然系统已经达到预期目标，但是仍然存在一些不足。例如推荐算法比较基础，缺少深度学习模型的支持。OCR 识别在处理复杂排版和手写内容时表现出来的能力比较有限。后续研究方向可以考虑运用比较先进的推荐算法，比如深度学习或者图模型；同时 OCR 识别精度可以考虑进行优化；完善自动化测试模式，以及提高系统的智能化和个性化服务能力。

致谢

时光荏苒，四年的大学生活即将画上句号。站在毕业的门槛上回望，这段旅程既有收获的喜悦，也有成长的磨砺。在此，我要向所有给予我帮助和支持的人致以最诚挚的谢意。

本毕业设计及论文是在王莉莉老师的悉心指导下完成的。从选题、系统设计到论文撰写的每一个环节，王老师都给予了我耐心的点拨和严谨的把关。每当我在技术实现或写作思路遇到困惑时，王老师总是不厌其烦地为我答疑解惑，并提出宝贵的修改意见。她治学严谨、待人亲和，不仅在学业上为我指明方向，也在生活中给予我温暖的鼓励。能遇到这样一位良师，我深感荣幸。

同时，感谢大学四年里每一位授课老师，是你们用生动的课堂带我不断深入计算机的世界，为我打下扎实的专业基础。感谢同窗的同学们，在无数个讨论和调试代码的日夜中，我们互相鼓励、共同进步。还要感谢我的家人，你们一直以来的理解与支持是我安心求学的最大动力。

最后，衷心祝愿王莉莉老师及所有老师工作顺利、桃李芬芳；祝愿同学们前程似锦，未来可期！

参考文献

- [1] 夏天生,苏俏媚,郭嘉跃. 基于国内网页端云笔记产品的可用性研究[J]. 包装工程,2025,46(20):445-455.
- [2] 刘春爽. Android 平台手写笔记系统的设计与实现[D].北京邮电大学,2022.
- [3] 沈骏. 基于文本相似度的个人笔记管理系统的设计与实现[D].华中科技大学,2020.
- [4] Li D L, Lee S K, Liu Y T.Printed document layout analysis and optical character recognition system based on deep learning[J]. Scientific Reports, 2025,15(1):23761.
- [5] Umar M, Sultana T, Lee Y K.Pre-trained language models for keyphrase prediction: A review[J]. KCI Journal,2024,10(4):871-890.
- [6] Canha H.Speed vs Accuracy: Tuning Elasticsearch Performance[EB/OL]. Syone,2025.
- [7] Woerndl W, Schueller C, Wojtech R. A Hybrid Recommender System for Context-aware Recommendations of Mobile Applications[C].IEEE 23rd International Conference on Data Engineering Workshop. Istanbul: IEEE,2007:871-878.
- [8] Thitiakkarasak P, Suriyo T, Jensawaspong S, et al. LoomLen: A Real-Time Collaborative Note-Taking Application for Mobile and Tablet Devices[C].Proceedings of King Mongkut's University of Technology Thonburi Conference. Bangkok: KMUTT,2025.
- [9] Sharkawi N, Mahmoud A, Tawalbeh A, et al.An AI-Based Interactive Digital Content Application for Document Summarization, Translation, and Productivity Enhancement[J]. Procedia Computer Science, 2025, 272: 187-194.
- [10] 邻安. 笔记软件上手对比：All-in-one 是不是伪命题？[N]. 电脑报,2024-06-24(012).
- [11] 隋烁.在线文档的内容节点组织研究[D].吉林大学,2023.
- [12] 李毅. 基于 Notion 的中国古代文学“三位一体双时段”教学模式探索[J].中国多媒体与网络教学学报(上旬刊),2022,(07):244-248.
- [13] 孙依琳. Notability 电子笔记软件用户体验研究[J]. 中国市场,2024,(07):60-63.

- [14] 郭星驰. 笔记软件“Obsidian”在个人数字存档中的应用研究——基于 DIKW 模型解释框架[J].无线互联科技,2025,22(02):96-100.
- [15] Suthar O P, Mishra A, Singhal S. Exploring the Landscape of Natural Language Processing for Text Analytics: A comprehensive Review[J]. Procedia Computer Science,2025,259:453-462.
- [16] 祝驿楠, 王世雄, 黄园. 基于 ROA 的移动个人知识管理系统集成[J]. 计算机系统应用, 2016, 25 (04): 278-282.
- [17] Kleppmann M, Beresford A R. A Conflict-Free Replicated JSON Datatype[J]. IEEE Transactions on Parallel and Distributed Systems, 2017, 28(10):2733-2746.

附录

英文原文

An AI-Based Interactive Digital Content Application for Document Summarization, Translation, and Productivity Enhancement

In the evolving digital era, managing extensive textual content remains a significant challenge for students, professionals, and researchers. This paper presents an AI-powered interactive digital content application that consolidates document summarization, translation, note-taking, to-do list management, and reading continuity into a unified mobile platform. Leveraging Google's Gemini API, the application delivers real-time content summarization and intelligent chatbot interactions. A Flutter-based frontend ensures a clean user experience, while a Flask backend and Firebase services support secure, cloud-synced document storage and user interaction. This paper outlines the system's architecture, implementation, and user-oriented features such as searchable PDFs, cross-language support, and task tracking. Performance tests validate the app's scalability, security, and usability. The proposed application bridges the gap between fragmented productivity tools by integrating AI with document handling in a mobile-first environment.

1. Introduction

In an age where information overload is prevalent, users often face challenges in managing, processing, and interacting with large volumes of digital documents. Traditional file readers and content management tools frequently fall short in offering intelligent support features such as summarization, content navigation, or productivity integration. This limitation not only affects the efficiency of students and professionals but also hampers equitable access to essential information.^[1] The integration of AI-driven

solutions can significantly enhance user engagement and facilitate more effective information retrieval from extensive documents, addressing the challenges posed by information overload.

The Application (app) presented in this paper proposes a novel, AI-enhanced digital content application that unifies several core features, including document summarization, AI chatbot assistance, translation services, searchable PDFs, and integrated productivity tools like note-taking and to-do lists. The application is built using Flutter for the frontend, Flask for the backend, and Firebase for secure, real-time cloud storage and authentication.^[2]

The core objective of this app is to reduce the cognitive and operational load associated with document handling by integrating AI-driven functionalities into a unified mobile-first platform. The application particularly targets users in academia and professional settings who require agile, responsive, and smart solutions for engaging with extensive digital content.^[3]

Unlike existing AI chatbots such as ChatGPT and DeepSeek, LAN offers a comprehensive, all-in-one platform specifically designed to support productivity and knowledge retention in a more integrated manner. While mainstream AI tools are optimized for conversational interactions, LAN goes beyond by embedding essential features such as built-in note-taking, to-do lists, bookmarking, and seamless PDF continuation; all of which are missing or limited in other tools. Additionally, LAN supports unlimited file uploads and API-based summarization, making it especially valuable for users managing extensive research, documentation, or study materials. With robust multilingual translation, built-in AI chatting, and deeply integrated content management capabilities, LAN is not just a chatbot; it is a versatile productivity companion purpose-built for continuous learning, critical thinking, and long-term engagement. This application aims to transform how users interact with digital content, making information retrieval and management more intuitive and efficient.

This paper explores the architectural components, implementation strategies, and user-centered design that guide the development of the application. The results demonstrate that the tool offers significant improvements in user productivity, content comprehension, and digital workflow efficiency.

2. Literature Review

2.1. AI-Based Summarization and Chatbots

Current research and literature surrounding the integration of artificial intelligence in productivity applications reveal a significant shortcoming: the absence of cohesive, unified systems. Summarization tools are frequently developed as isolated services, lacking seamless connectivity with other functionalities. Meanwhile, chatbots have made remarkable strides in natural language understanding; however, they are seldom incorporated within document processing applications, limiting their utility in enhancing user experience. The proposed system seeks to address this critical gap by directly embedding Google's Gemini API into the workflow, allowing for a more integrated and efficient approach to document summarization and interactive assistance.^{[4][5]} This integration not only enhances the summarization capabilities but also allows for real-time interactions that can significantly improve user engagement and productivity

2.2. Document Management and Search

While established applications such as Adobe Acrobat provide basic annotation and search functionalities, they fall short in offering persistent session memory and seamless integration with other productivity tools. The proposed system introduces several innovative features, including real-time PDF search capabilities, bookmarking options, and the ability to continue sessions without interruption. These enhancements significantly reduce friction in document engagement, enabling users to navigate, annotate, and retrieve

information more fluidly and effectively.

2.3. Translation and Multilingual Support

Despite the widespread adoption of the Google Translate API, its implementation within note-taking and content management systems remains scarce. The proposed application addresses this oversight by integrating a user-friendly translation widget directly into the interface. This feature allows users to effortlessly switch between languages while maintaining their workflow, thereby enhancing accessibility and collaboration in multilingual environments. This seamless integration ensures that language barriers do not hinder productivity, making the tool more versatile and user-centric.^[6] The application's design emphasizes user-centric features that facilitate smooth transitions between summarization, translation, and task management, ultimately fostering a more efficient digital workspace.

2.4. Productivity Tools Integration

Many applications that provide note-taking and task management functionalities often lack the integration of AI assistance or advanced document search capabilities. The proposed tool overcomes this fragmentation by storing all user data in^[10] base, which facilitates unified access through a clean and intuitive user interface^[7]. This holistic approach not only streamlines the user experience but also enhances the overall efficiency of managing tasks and notes, ensuring that users have all the resources they need at their fingertips without the hassle of navigating multiple disjointed systems.^[8]

The literature review compellingly demonstrates a significant market demand for multifunctional productivity platforms that seamlessly integrate artificial intelligence with digital content tools. This initiative aims to bridge that gap by creating a mobile-centric application that is not only exceptionally user-friendly and feature-rich but also empowered by cutting-edge artificial

intelligence capabilities. By tackling the existing shortcomings in current solutions, this revolutionary platform aims to dramatically boost productivity and significantly enhance user engagement in an increasingly digital work landscape.

3. System Design and Architecture

The system adopts a modular, layered architecture that ensures maintainability, scalability, and performance across various mobile platforms. This architecture enables efficient updates and integration of new features, ensuring that the application remains responsive to user needs and technological advancements.

- 3.1. System Overview
- 3.2. Functional Components
- 3.3. Security and Access Control
- 3.4. Data Architecture
- 3.5. User Experience Design

4. Software Implementation

The LAN application is designed using a modular architecture that separates concerns across presentation, application logic, and data layers. Built with Flutter for the frontend and Flask for the backend, the system leverages Firebase for authentication and storage, and integrates with Google's Gemini API for AI functionalities.

- 4.1. Chatbot Component
- 4.2. Summarization Component
 - 4.2.1. Frontend (Flutter)
 - 4.2.2. Service Layer
 - 4.2.3. Backend (Flask)
- 4.3. Translation Component
- 4.4. Notes Component
- 4.5. To-Do List Component

4.6. Add File Component

4.7. Bookmark & Continue Reading

5. Testing and Validation

The validation process confirmed that the application fulfilled its functional requirements. A series of essential tests were carried out to verify performance, security, and usability. Functional Testing concentrated on key features such as PDF upload, AI-driven summarization, AI chatbot responses, and the note-taking capability. Test documents were utilized to verify the precision and dependability of the summaries generated by AI. Performance Testing validated the app's responsiveness across various devices. Security Testing confirmed that Firebase Authentication functioned correctly, safeguarding user data and preventing unauthorized access. The outcomes of the testing verified that the application functioned as intended, satisfying all design specifications and providing a seamless user interface

6. Discussion

The application that was developed successfully met its main goals, providing a powerful, AI-boosted digital content management system designed for enhanced productivity and user engagement. By incorporating Google's Gemini API, the app offers quick, context-aware summarization, greatly enhancing content understanding and minimizing reading duration. The addition of a conversational chatbot further enhances usability, providing intuitive, real-time assistance that streamlines document interaction and promotes a more natural digital reading experience. Core document management functions—like uploading, reading, and searching PDFs—were crafted with a focus on effectiveness. The capability for searchable PDFs allows users to easily find and highlight important terms, thus improving information accessibility.

Auxiliary features such as note-taking and to-do lists, which are synchronized in real-time through Firebase, help create a more organized and task-oriented experience. The “Continue Reading” and bookmarking features

enhance document navigation, enabling users to effortlessly pick up where they left off and sustain their reading context. From a security perspective, the implementation of Firebase Authentication and Cloud Firestore guarantees the integrity of user data, real-time persistence, and a safe environment for the management of personal or sensitive information. In summary, the system integrates AI-driven intelligence with vital productivity tools within a cross-platform, scalable architecture. It has demonstrated itself to be a valuable resource for e-reading and knowledge management, with a well-defined path for future advancements.

7. Conclusion

This app set out to develop an AI-powered, interactive application to enhance the digital reading experience by integrating intelligent tools such as summarization, translation, chatbot assistance, and task management. Utilizing Flutter for the frontend, Flask for backend APIs, and Firebase for authentication and storage, the system successfully meets the goals of performance, scalability, and user-centered design. The integration of Gemini's large language model enabled fast, coherent summaries and intelligent conversational support. Users benefited from an immersive and efficient reading workflow—supported further by intuitive UI design, real-time data syncing, and cross-functional utilities such as note-taking and to-do list management.

Based on our implementation experience and user feedback, the following enhancements are strongly recommended to guide future versions:

1. Improve AI Accuracy: Employ more advanced NLP models to refine the summarization and chatbot response quality.
2. Widen Translation Support: Expand language options to accommodate a broader international audience.
3. Enable Offline Mode: Offer offline access to summaries, notes, and PDFs to support users in low-connectivity environments.
4. Introduce Advanced Search: Incorporate keyword suggestions, filters,

and voice-enabled search for quicker content retrieval.

5. Integrate User Feedback Tools: Build an in-app mechanism for feedback, bug reporting, and suggestions.

6. Explore Video Summarization: Leverage AI to summarize video content, broadening the application's utility.

The proposed application not only addresses the challenges of document management but also sets a precedent for future innovations in AI-driven productivity tools. In conclusion, the proposed application showcases the potential of AI to transform digital content management, offering users a comprehensive suite of tools for enhanced productivity and engagement.

中文译文

基于人工智能的交互式数字内容应用，用于文档摘要生成、文本翻译及生产力提升

在不断发展的数字时代，处理海量文本内容对学生、专业人士和研究人员而言仍是一项重大挑战。本文介绍了一款基于人工智能的交互式数字内容应用，该应用将文档摘要生成、文本翻译、笔记记录、待办事项管理及阅读连贯性等功能整合到统一的移动平台上。通过调用谷歌 Gemini API，该应用可实现实时内容摘要生成和智能聊天机器人交互。基于 Flutter 的前端确保了简洁的用户体验，而 Flask 后端与 Firebase 服务则支持安全、云端同步的文档存储及用户交互。本文详细阐述了系统的架构、实现方案以及以用户为中心的功能特性，包括可搜索 PDF 文档、多语言支持和任务追踪功能。性能测试验证了该应用的可扩展性、安全性及易用性。所提出的应用通过在移动优先环境中将人工智能与文档处理技术深度融合，有效弥合了现有分散式生产力工具之间的差距。

1. 引言

在信息过载普遍存在的当今时代，用户在管理、处理和操作海量数字文档时常常面临诸多挑战。传统的文件阅读器和内容管理工具往往无法提供摘要生成、内容导航或生产力集成等智能支持功能。这一局限不仅影响了学生和专业人士的工作效率，还阻碍了对关键信息的公平获取。引入人工智能驱动的解决方案能够显著提升用户参与度，并帮助从海量文档中更

高效地检索信息，从而有效应对信息过载带来的挑战。

本文介绍的应用程序（app）是一款创新的、基于人工智能增强的数字内容应用，集成了多项核心功能：文档摘要生成、人工智能聊天机器人辅助、翻译服务、可搜索 PDF 文件，以及笔记记录和待办事项列表等集成化生产力工具。该应用采用 Flutter 构建前端界面，Flask 构建后端系统，并运用 Firebase 实现安全的实时云存储与身份验证功能。

这款应用程序的核心目标是通过将人工智能驱动的功能整合到一个统一的移动优先平台上，减轻文档处理相关的认知负担和操作负担。该应用主要面向学术界和专业领域的用户，他们需要能够灵活、快速响应且智能化的解决方案来处理海量数字内容。

与 ChatGPT 和 DeepSeek 等现有 AI 聊天机器人不同，LAN 提供了一个全面的一体化平台，专为以更整合的方式提升生产力和知识留存而设计。主流 AI 工具主要针对对话交互进行优化，而 LAN 则更进一步——它内置了笔记记录、待办事项列表、书签功能以及无缝 PDF 续读等核心功能，这些功能在其他工具中要么缺失要么功能有限。此外，LAN 支持无限文件上传和基于 API 的摘要生成，对于需要管理大量研究资料、文档或学习材料的用户而言尤为实用。凭借强大的多语言翻译能力、内置 AI 聊天功能以及深度集成的内容管理功能，LAN 不仅仅是一个聊天机器人，更是一个专为持续学习、批判性思维和长期互动而打造的多功能生产力助手。该应用旨在彻底改变用户与数字内容的交互方式，使信息检索与管理变得更加直观高效。

本文探讨了指导该应用程序开发的架构组件、实现策略及以用户为中心的设计理念。研究结果表明，该工具在提升用户生产力、增强内容理解能力以及优化数字工作流程效率方面均取得了显著改进。

2. 文献综述

基于人工智能的摘要生成与聊天机器人。当前关于人工智能在生产力应用中整合的研究与文献揭示了一个显著缺陷：缺乏连贯统一的系统架构。摘要生成工具通常被开发为独立的服务模块，与其他功能之间缺乏无缝衔接。与此同时，聊天机器人在自然语言理解方面取得了显著进展，但它们很少被整合到文档处理应用中，这限制了其在提升用户体验方面的效用。本系统旨在通过将谷歌 Gemini API 直接嵌入工作流程来解决这一关键问题，从而实现更集成、更高效的文档摘要生成与交互式辅助功能。这种集成不仅提升了摘要生成能力，还支持实时交互，能显著提高用户参与度与工作效率。

文件管理与检索。虽然 Adobe Acrobat 等成熟应用程序提供了基本的标注和搜索功能，但在提供持久会话内存以及与其他生产力工具无缝集成方面仍存在不足。该系统引入了多项创新功能，包括实时 PDF 搜索能力、书签选项以及无需中断即可继续会话的功能。这些改进显著降低了文档操作

过程中的障碍，使用户能够更流畅、高效地进行文档浏览、标注和信息检索。

翻译与多语言支持。尽管谷歌翻译 API 已被广泛采用，但在笔记记录和内容管理系统中的实际应用仍较为匮乏。本应用通过将用户友好的翻译小工具直接集成到界面中，有效弥补了这一不足。该功能让用户能够在保持原有工作流程的同时轻松切换语言，从而提升多语言环境下的使用便捷性与协作效率。这种无缝集成确保语言障碍不会影响工作效率，使工具更具多功能性和用户中心性。该应用的设计重点突出以用户为中心的功能，实现了摘要生成、翻译处理与任务管理之间的无缝衔接，最终打造出更高效的数字工作空间。

生产力工具集成。许多提供笔记记录和任务管理功能的应用程序往往缺乏人工智能辅助或高级文档搜索功能的集成。本工具通过将所有用户数据存储在基础数据库中，克服了这种碎片化问题，用户可通过简洁直观的界面实现统一访问。这种整体性设计不仅优化了用户体验，还提升了任务和笔记管理的整体效率，让用户无需在多个分散的系统间来回切换，即可轻松获取所需的所有资源。

文献综述有力地表明，市场对能够无缝整合人工智能与数字内容工具的多功能生产力平台存在显著需求。本项目旨在通过开发一款以移动端为核心的应用程序来填补这一空白——该应用不仅具备极高的用户友好性和丰富的功能，更依托前沿的人工智能技术实现强大功能。通过解决现有解决方案中的不足之处，这一革命性平台有望在日益数字化的工作环境中显著提升生产力，并大幅增强用户的参与度。

3. 系统设计与架构

该系统采用模块化、分层的架构设计，确保了在各种移动平台上具备良好的可维护性、可扩展性及性能表现。这种架构能够高效地实现功能更新与新功能集成，从而使应用程序始终能够及时响应用户需求和技术发展。

3.1. 系统概述

3.2. 功能组件

3.3 安全与访问控制

3.4 数据架构

3.5 用户体验设计

4. 软件实现

该局域网应用采用模块化架构设计，将系统功能划分为展示层、应用逻辑层和数据层三个独立模块。前端使用 Flutter 开发，后端采用 Flask 构建；系统利用 Firebase 实现身份验证与数据存储，并通过集成 Google Gemini API 实现人工智能功能。

4.1. 聊天机器人组件

4.2. 系统组件

4.2.1. 前端（Flutter）

4.2.2 服务层

4.2.3. 后端（Flask）

4.3. 翻译组件

4.4. 注释组件

4.5. 待办事项列表组件

4.6. 添加文件组件

4.7. 书签与继续阅读组件

5. 测试与验证

验证流程证实该应用程序已满足其功能要求。我们开展了一系列关键测试以验证性能、安全性和易用性。功能测试重点考察了 PDF 上传、人工智能驱动的摘要生成、AI 聊天机器人响应以及笔记记录等功能。通过测试文档验证了 AI 生成摘要的精确度与可靠性。性能测试验证了应用程序在不同设备上的响应速度。安全测试确认 Firebase 身份验证功能运行正常，有效保护用户数据并防止未经授权的访问。测试结果表明，该应用程序按预期运行，完全符合所有设计规范，并提供了流畅的用户界面。

6. 讨论

这款成功开发的应用程序实现了其主要目标，提供了一套功能强大、基于人工智能优化的数字内容管理系统，旨在提升工作效率和用户参与度。通过整合谷歌 Gemini API，该应用具备快速、基于上下文的摘要功能，显著增强了内容理解能力并缩短了阅读时间。新增的对话式聊天机器人进一步提升了用户体验，提供直观的实时协助，简化了文档操作流程，营造出更自然的数字阅读体验。核心文档管理功能——如上传、阅读和搜索 PDF 文件——均以高效性为核心设计。支持 PDF 文件搜索的功能让用户能轻松查找并标注重要术语，从而提升信息获取的便捷性。

诸如笔记记录和待办事项列表等辅助功能，通过 Firebase 实时同步，有助于打造更加井然有序、以任务为导向的使用体验。“继续阅读”和书签功能提升了文档导航效率，让用户能够轻松从上次阅读位置继续，并保持阅读连贯性。从安全角度来看，Firebase 身份验证和 Cloud Firestore 的应用确保了用户数据的完整性、实时持久性，以及个人或敏感信息管理的-safe 环境。总之，该系统将人工智能驱动的智能功能与关键生产力工具整合到一个跨平台、可扩展的架构中。它已被证明是电子阅读和知识管理的宝贵工具，并为未来的发展指明了清晰方向。

7. 结论

这款应用程序旨在开发一款基于人工智能的交互式应用，通过整合摘要生成、翻译功能、聊天机器人辅助及任务管理等智能工具，提升数字阅

读体验。系统采用 Flutter 构建前端界面，Flask 处理后端 API 接口，并运用 Firebase 实现身份验证与数据存储，成功实现了高性能、可扩展性及用户中心化设计的目标。借助 Gemini 大型语言模型的集成，应用能够生成快速、连贯的摘要并提供智能对话支持。用户可享受沉浸式且高效的阅读流程——这一体验还得到了直观的用户界面设计、实时数据同步功能，以及笔记记录和待办事项管理等跨功能工具的有力支持。

根据我们的实施经验及用户反馈，强烈建议对以下功能进行改进，以指导未来版本的开发：

7. 提升 AI 准确性：采用更先进的 NLP 模型，以优化摘要生成和聊天机器人响应质量。

8. 扩大翻译支持范围：增加语言选项，以满足更广泛的国际用户需求。

9. 启用离线模式：提供摘要、笔记和 PDF 文件的离线访问功能，以支持网络连接条件较差的用户使用。

10. 引入高级搜索功能：支持关键词建议、筛选条件及语音搜索，可更快地查找内容。

11. 整合用户反馈工具：在应用内构建用于收集反馈、报告漏洞及提出建议的功能模块。

12. 探索视频摘要功能：利用人工智能对视频内容进行摘要，从而提升该应用的实用性。

该提案的应用方案不仅解决了文档管理中的各项挑战，更为未来人工智能驱动型生产力工具的创新树立了先例。总之，该方案充分展现了人工智能在数字化内容管理领域的变革潜力，为用户提供了全面的工具组合，从而显著提升工作效率与用户参与度。